



School of Art and Creative Technologies

M.Sc. Artificial Intelligence Computing

University of Bolton

Deciphering Cyberbullying: Enhancing Sentiment Analysis with Explainable AI Techniques

Prepared by Usama Saleem

Student ID: 2336804

Supervisor: Celestine Iwendi

A report submitted in fulfillment for the MSc degree in Artificial Intelligence Computing

April 2025

The University of Bolton

Dean Road, Bolton, BL3 5AB

<https://www.bolton.ac.uk>

DEDICATION

To Allah Almighty, who is the source of all wisdom and the one who granted me strength and perseverance to complete this work.

ACKNOWLEDGEMENT

Firstly, all thanks and praise are to Almighty Allah, the Most Merciful and Benevolent, for giving me the strength, wisdom, and patience to complete this research. Without His many favors, this work would never have been completed. I am highly indebted to my supervisor, Professor Celestine Iwendi, for the direction, forbearance, and unwavering encouragement he offered during this research. His know-how, helpful critiques, and support were instrumental in developing this work. I also sincerely thank my dear friend Farhan, whose unflinching support, unbreakable encouragement, and stimulating debates have helped me overcome many challenges. I thank my cousin Adnan for his constant support and encouragement in adversity. Finally, I extend my sincerest appreciation to my family in Pakistan—parents, siblings, and dear ones—for their prayers and unconditional love. Their belief in me has been my biggest source of strength. This success belongs to them as much as it belongs to me.

ABSTRACT

Cyberbullying has become a widespread problem in digital communication, negatively affecting people's psychological health, particularly that of teenagers. Serious difficulties for automated detection systems arise from the growing complexity of online harassment, which is fuelled by sarcasm, coded expressions, and dynamic language change. This study explores the use of deep learning architectures (Bi-LSTM, GRU, DistilBERT) and conventional machine learning models (Logistic Regression, Random Forest, SGD) to identify cyberbullying content on social media platforms. The study incorporates Explainable AI (XAI) techniques—SHAP (Shapley Additive Explanations), LIME (Local Interpretable Model-Agnostic Explanations), and attention visualization—to improve model transparency and build confidence in light of the drawbacks of black-box deep learning models. Results from experiments show that deep learning models perform better than conventional techniques; Bi-LSTM had the highest F1-score of 86%, closely followed by DistilBERT. But when combined with XAI methods, machine learning models provide better interpretability, which makes them useful for applications that need human intervention right away. Analysing explainability provided important information about model behaviour, possible biases, and places where ethical risk may be reduced. This thesis emphasises how explainability and predictive performance must be combined in cyberbullying detection systems to provide responsibility, fairness, and user confidence. In order to effectively address the changing panorama of online harassment, it finishes by highlighting the necessity of dynamic retraining, multilingual adaption, and human-in-the-loop moderation systems.

Keywords: Machine Learning, Deep Learning, Explainable AI, Transparency, Interpretability, Cyberbullying Detection, LIME, SHAP, Attention Visualization.

TABLE OF CONTENTS

DEDICATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	viii
LIST OF TABLES	ix
1. INTRODUCTION	1
1.1. Background of Study	1
1.1.1. Cyberbullying Epidemic	1
1.2. Problem Statement	5
1.3. Research Questions	7
1.4. Research Contribution to Knowledge	8
1.5. Aim and Objectives of the Research	8
1.5.1. Aim	8
1.5.2. Objectives	9
1.6. Scope of this Study	9
1.7. Thesis Organization	10
2. LITERATURE REVIEW	12
2.1. Existing Cyberbullying Detection Models	12
2.1.1. Support Vector Machines (SVM)	12
2.1.2. Naive Bayes with User Metadata	13
2.1.3. Decision Trees & Rule-Based Systems	14
2.2. Deep Learning Cyberbullying Detection Models	16
2.2.1. Convolutional Neural Networks (CNNs)	16
2.2.2. Long Short Term Memory (LSTM)	17
2.2.3. Transformer Models (BERT, RoBERTa)	18
2.2.4. Deep Learning Models Criticism	20
2.3. Challenges in Cyberbullying Detection	22
2.4. Explainable AI (XAI) in Cyberbullying Detection	23
2.5. Literature Review Summary Table	25

2.6.	Literature Review Summary	38
3.	RESEARCH METHODOLOGY	40
3.1.	Research Design	40
3.2.	Experimental Framework.....	40
3.3.	Dataset & Preprocessing.....	40
3.3.1.	Data Collection.....	40
3.3.2.	Preservation of Context	41
3.3.3.	Data Preprocessing	41
3.4.	Model Development	41
3.4.1.	Machine Learning Models	41
3.4.2.	Deep Learning Models	43
3.5.	Explainability Techniques.....	45
3.5.1.	LIME (Local Interpretable Model-Agnostic Explanations)	45
3.5.2.	SHAP (SHapley Additive exPlanations)	46
3.5.3.	Attention Mechanisms.....	47
3.6.	Model Evaluation	47
4.	RESULTS & IMPLEMENTATION	50
4.1.	Model Performance Analysis	50
4.2.	Machine Learning Model Performance	50
4.3.	Deep Learning Model Performance.....	51
4.4.	Explainability Results	55
4.4.1.	LIME Analysis in Machine Learning Models.....	55
4.4.2.	SHAP Analysis in Machine Learning Models	59
4.4.3.	LIME Analysis in Deep Learning Models	62
4.4.4.	SHAP Analysis in Deep Learning Models.....	65
4.4.5.	Attention Visualization Analysis for DistilBERT Model	68
4.5.	Results & Implementation Summary.....	69
5.	DISCUSSIONS.....	71
5.1.	Overview	71
5.2.	Machine Learning Models Discussion.....	71
5.3.	Deep Learning Models Discussion	72
5.4.	Explainable AI (XAI) Techniques Discussion.....	72

5.5.	Ethical & Practical Implications.....	73
5.6.	Limitations of Study	74
5.7.	Discussions Summary.....	74
6.	CONCLUSION & FUTURE WORK.....	76
6.1.	Conclusion.....	76
6.2.	Limitations.....	77
6.3.	Future Work	77
7.	REFERENCES	79
8.	APPENDICES	83
8.1.	Gantt Chart.....	83
	Implementation File 1 Code Snippets.....	83
	Implementation File 2 Code Snippets.....	104

LIST OF FIGURES

Figure 1.1: How Explainable AI (XAI) enhances cyberbullying detection	4
Figure 1.2: Venn Diagram (Traditional Models VS XAI Enhanced Models)	7
Figure 4.1: Machine Learning Models Performance Bar Chart	51
Figure 4.2: Deep Learning Models Performance Bar Chart	53
Figure 4.3: BI-LSTM Confusion Matrix	54
Figure 4.4: Machine Learning LIME Explanations Case 1	55
Figure 4.5: Machine Learning LIME Explanations Case 2	56
Figure 4.6: Machine Learning LIME Explanations Case 3	57
Figure 4.7: Machine Learning LIME Explanations Case 4	58
Figure 4.8: SHAP Explanations for Logistic Regression	59
Figure 4.9: SHAP Explanations for Random Forest	60
Figure 4.10: SHAP Explanations for SGD Classifier	61
Figure 4.11: LIME Explanations for Deep Learning Models	62
Figure 4.12: BI-LSTM - LIME Analysis	63
Figure 4.13: GRU - LIME Analysis	63
Figure 4.14: DistilBERT - LIME Analysis	64
Figure 4.15: SHAP Explanations for Deep Learning Models	65
Figure 4.16: BI-LSTM - SHAP Analysis	66
Figure 4.17: GRU - SHAP Analysis	66
Figure 4.18: DistilBERT - SHAP Analysis	67
Figure 4.19: DistilBERT - Attention Visualization	68
Figure 4.20: DistilBERT - Attention Analysis	69
Figure 8.1: Thesis Gantt Chart	83

LIST OF TABLES

Table 2.1: Summary of Literatures	25
Table 4.1: Machine Learning Models Performance Metrics	50
Table 4.2: Deep Learning Model Performance Metrics.....	52

1. INTRODUCTION

The growth of social media sites has revolutionised global communication; however, it has also resulted in a worrying rise in cases of cyberbullying. Recent studies indicate that 59% of American adolescents have been victims of online harassment, with 14% suffering severe psychological damage, including depression and suicidal tendencies (Pew Research Centre, 2023). Despite the advancements in machine learning (ML) and natural language processing (NLP) techniques, their "black-box" nature reduces trust and accountability of content moderation systems. This research bridges the large gap between detection performance and interpretability by integrating Explainable AI (XAI) techniques, thus encouraging open and ethically sound solutions towards online safety.

1.1. Background of Study

1.1.1. Cyberbullying Epidemic

Cyberbullying is characterised as deliberate, repeated aggression via electronic media with different features than conventional bullying:

Permanence: The offensive content is available forever.

Amplification: Thousands may see one post within minutes.

Anonymity: Perpetrators tend to use pseudonyms.

Psychological Effects: Victims have a 3× higher risk of depression compared to non-victims. Suicide for adolescents who were cyberbullied grew by 45% between 2015 and 2022 (CDC, 2025). The issue of cyberbullying has been deeply discussed to bring light to the fact that how much a victim may have been affected by it: mental health that results in physical and emotional devastation, low self-esteem, subpar performance, or other issues. As more and more people are getting connected through social networking sites

and other online communication channels, the incidents of online harassment, hate speech, and cyber threats have also increased. Cyberbullies are more likely to report higher levels of violence, substance abuse, and delinquent behaviour (Nixon, 2014).

For teenage victims who were defeated and trapped by cyberbullying and their pessimistic attitudes, participants considered that suicide was a feasible way out. Research indicates that adolescents subjected to persistent cyberbullying are at a significantly higher risk of suicidal ideation and self-harm due to feelings of helplessness and social isolation. Most victims of cyberbullying contemplate suicide as an escape from the psychological torture they are exposed to, with studies showing that cyberbullying victims are twice as likely to attempt suicide compared to non-victims. It is, therefore, very critical that there should be effective intervention and detection mechanisms to help alleviate the effects caused by cyberbullying (Dennehy et al., 2020). Early detection and psychological support systems are essential in mitigating long-term mental health consequences (Nixon, 2014).

In resolving this problem, machine learning (ML) and natural language processing (NLP) have been extensively applied to create automated models for detecting cyberbullying. Transformer-based models like BERT and RoBERTa have recently been found to be better at detecting subtle and context-specific harassment (Devlin et al., 2019). The models have been found to have high precision in detecting offensive and malicious messages. This lack of transparency is a major cause for concern in terms of bias, fairness, and accountability, especially when automatic systems flag or fail to identify dangerous content (Mehrabian et al., 2021). For example, biased training data may lead models to disproportionately target specific demographic segments, amplifying inherent societal inequalities. To counter these, more recent research recommends the use of explainable AI (XAI) methods, including SHAP (Shapley Additive Explanations) and LIME (Local Interpretable Model-agnostic Explanations), to increase transparency in cyberbullying identification systems (Lundberg & Lee, 2017; Ribeiro et al., 2016).

However, one of the biggest shortcomings of these approaches is that they are not very interpretable. The majority of deep learning models are black-box systems, which means it is hard to explain how decisions are made. This makes it very concerning in terms of bias, fairness, and accountability, especially when automated systems falsely mark or fail to mark damaging content (Mehrabi et al., 2021). To solve such issues, new research recommends using explainable AI (XAI) methods like SHAP (Shapley Additive Explanations) and LIME (Local Interpretable Model-agnostic Explanations) to provide more transparency to cyberbullying detection systems (Lundberg & Lee, 2017; Ribeiro et al., 2016).

Enhancing Sentiments Analysis with Explainable AI Techniques

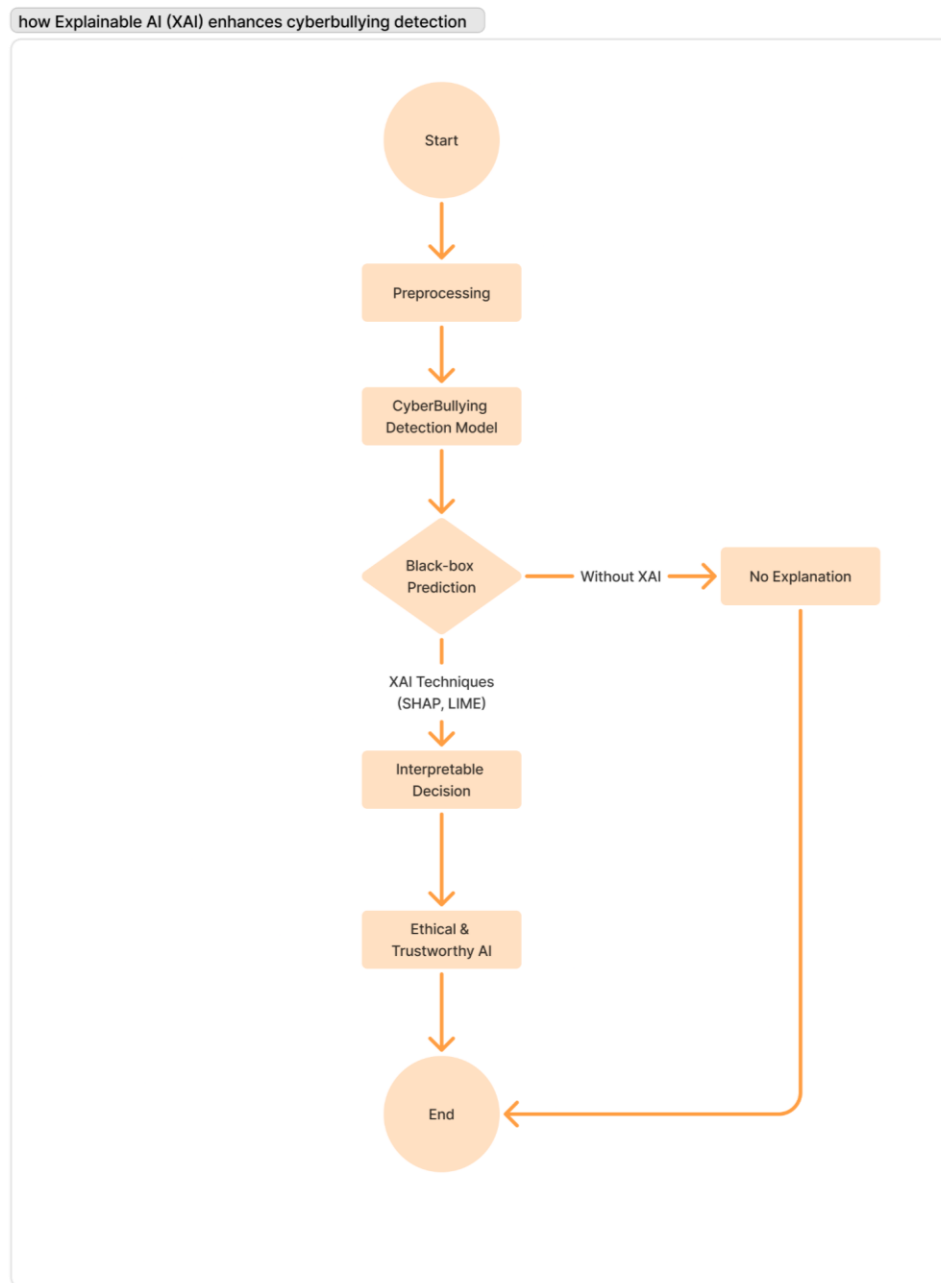


Figure 1.1: How Explainable AI (XAI) enhances cyberbullying detection

Figure 1.1 shows our flowchart which will be implemented to improve the transparency and interpretability of traditional machine learning and deep learning models' decision-making processes. Explainable AI (XAI) seeks to overcome their "black-box" nature. Researchers, legislators, and platform stakeholders can examine predictions to make sure

they comply with legal and ethical requirements thanks to XAI's model explainability (Doshi-Velez & Kim, 2017).

To improve interpretability in cyberbullying detection systems, several XAI techniques have been used. These consist of:

To help identify important indicators of cyberbullying, SHapley Additive exPlanations (SHAP) measures the contribution of each input feature (such as particular words or phrases) to a model's prediction (Lundberg & Lee, 2017).

By altering input data and tracking changes in output, Local Interpretable Model-Agnostic Explanations (LIME) produces condensed, comprehensible explanations for individual predictions (Ribeiro et al., 2016).

In Transformer Models, attention visualisation shows which textual elements—such as abusive keywords or contextual cues—the model considers when identifying content as bullying.

In addition to increasing model transparency, these techniques assist in examining systems for fairness and bias to make sure minorities are not disproportionately targeted (Mehrabi et al., 2021).

While these models are highly accurate, their interpretability issues prevent them from being used in content moderation systems in the real world, where false positives, such as misclassifying sarcasm as bullying, can have adverse impacts. By creating interpretable yet trustworthy models that can be successfully used in online safety initiatives, the integration of XAI fills this gap.

1.2. Problem Statement

The growing problem of cyberbullying on digital platforms has made content moderation systems face difficult problems. Because of this opacity in decision-making, there is a paradox whereby systems intended to improve online safety could unintentionally cause harm through the following measures:

Operational Restrictions in Practical Implementation:

Contextual nuance is a challenge for current systems, which frequently fail to differentiate between benign conversations with similar lexical patterns and malicious intent. Bullies and detection systems engage in an "arms race" due to the dynamic nature of online communication, which includes changing slang and coded language.

Algorithmic Moderation's Effect on the Mind:

When valid reports are rejected without justification, opaque moderation decisions have the potential to retraumatize victims. Particularly in vulnerable populations, the "chilling effect" of false positives can discourage genuine self-expression.

Systemic Challenges with Platform Governance:

As platforms are sued for both excessive censorship and insufficient moderation, legal experts observe growing liability concerns. Comparative evaluation of various detection techniques is challenging due to the absence of standardised evaluation frameworks (Van Hee et al., 2018)..

The opportunity to turn cyberbullying detection from a purely technical problem into a multidisciplinary solution that:

Complies with psychological best practices for moderating trauma-informed content, satisfies changing legal requirements for algorithmic transparency, preserves computational effectiveness while offering decision trails that can be audited. Based on the expanded framework, the next generation of detection systems will need to strike a balance between three incompatible priorities: psychological sensitivity, technical

accuracy, and legal compliance. This will require a fundamental rethink of how we develop and assess these vital safety tools.

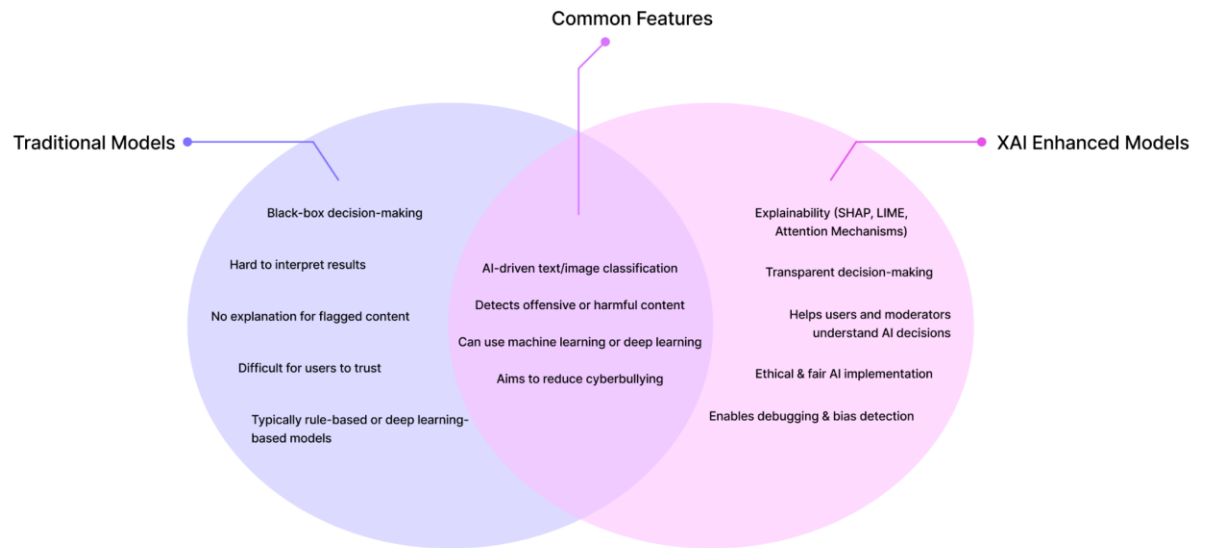


Figure 1.2: Venn Diagram (Traditional Models VS XAI Enhanced Models)

As we see in Figure 1.2, we will close this gap in the current study by creating and assessing XAI-enhanced cyberbullying detection models that ensure interpretability with high accuracy.

1.3. Research Questions

- 1) To what extent can current detection techniques identify instances of cyberbullying on social media?
- 2) What are the interpretability and transparency shortcomings of the models in use today?
- 3) How may attention mechanisms, LIME, SHAP, and other XAI techniques be used to improve the interpretability of cyberbullying detection models?

- 4) What effects does explainability have on decision-making, user trust, and model effectiveness in cyberbullying detection?
- 5) In what ways can integrating XAI techniques support the ethical application of AI in social media monitoring?

1.4. Research Contribution to Knowledge

This research contributes to knowledge in the development of an interpretable model of cyberbullying detection using Explainable AI techniques. It does this by enhancing transparency, integrating methods such as SHAP, LIME, and attention visualization to make AI decisions more interpretable, unlike in traditional black-box machine learning models. This is also done to ensure equity in the models by assessing the different biases within the models in the detection systems for ethical online content moderation. It also presents a comparative performance review of the models that would help optimize NLP-based cyberbullying detection. Finally, the study offers practical recommendations to policymakers, educators, and social media on how to implement equitable and trustworthy AI-driven moderation systems.

1.5. Aim and Objectives of the Research

1.5.1. Aim

The key objective of this research is the proposition of an interpretable cyberbullying detection model using machine learning and explainable AI techniques. This work is informed by the goal of making sure that cyberbullying detection systems are transparent, fair, and accountable; thus, practical and ethical for application in real-world settings. In this work, XAI methods will be used to increase the trustworthiness of AI-

driven content moderation and bridge the gap between high-performing machine learning models and their applicability for digital safety.

1.5.2. Objectives

The specific objectives of this study in achieving the stated aim are as follows:

- To review the existing methods for cyberbullying detection and their limitations, thus finding the gaps in interpretability and fairness.
- The main objective of this research was to develop machine learning models for the detection of cyberbullying using advanced NLP techniques.
- Techniques of explainability: SHAP, LIME, and visualization of attention shall be implemented for improved explainability and transparency in models.
- The resultant models developed will be evaluated based on appropriate performance metrics, which include accuracy, precision, recall, and interpretability.
- It would consider the impact that Explainable AI makes to mitigate bias in favour of fairness in detecting cyberbullying.
- It would also provide concrete recommendations for the practical integration of fair and interpretable AI-driven systems in cyberbullying detection and online content moderation.

1.6. Scope of this Study

The focus of this thesis encompasses:

1. Elaborates on developing an interpretable cyberbullying detection model through Explainable AI techniques.

2. Employs machine learning and NLP to understand publicly available datasets related to cyberbullying.
3. Explainability techniques that have been implemented to enhance model transparency include SHAP, LIME, and attention visualization.
4. Standard performance metrics of the model include accuracy, precision, recall, and interpretability.
5. Fairness issues are identified and mitigated in the cyberbullying detection models.
6. Allows policymakers, instructors, and online portals to create balanced AI-generated content moderation mechanisms.
7. It presents a weak point: the availability of the dataset, linguistic richness, and computational limitations sometimes restrict it further.

1.7. Thesis Organization

This thesis is organised as follows:

1. Chapter One: Introduction: This chapter states the background, statement of problem, aims, objectives, research questions, significance, scope, and structure of the thesis.
2. Chapter Two: Literature Review: This chapter shall review previous studies related to the detection of cyberbullying, machine learning techniques, approaches to NLP, and Explainable AI to highlight lapses and therefore establish the justification of the research study.
3. Chapter Three: Research Design and Methods: This chapter details the research design, dataset selection, preprocessing techniques, model development, explainability techniques (SHAP, LIME, attention visualization), and evaluation metrics used in the study.

4. Chapter Four: Implementation, Results, and Analysis: This chapter elaborates on experimental results and evaluation of model performances, and discusses the influence of explainability techniques on the transparency, fairness, and interpretability of models.
5. Chapter Five: Conclusion, Limitations, and Future Work: The final chapter summarizes the research findings and discusses the knowledge contribution and limitations. Further recommendations are made on future research directions in explainable AI-based cyberbullying detection and ethical AI applications

2. LITERATURE REVIEW

2.1. Existing Cyberbullying Detection Models

Early efforts in cyberbullying detection relied on traditional machine learning models, including Naïve Bayes classifiers, decision trees, and support vector machines (SVM) (Hasan et al., 2023). To create efficient classifiers, these models mainly relied on manual feature engineering, in which researchers chose and extracted characteristics such as syntactic structure, linguistic tone, sentiment scores, and user demographics (Chen, Wang and He, 2024). Using pre-established rules and keyword patterns, these features were used to detect harmful behaviour (Hasan et al., 2023). These early systems were easy to understand, quick to train, and comparatively simple. But they struggled with generalisation, especially when faced with the linguistic intricacy and inventiveness of online discourse (Chen, Wang and He, 2024). The static nature of these models soon became a drawback as online platforms developed to incorporate more complex forms of expression, including regional slang, emojis, memes, and sarcasm. The robustness of cyberbullying detection efforts was undermined by their frequent false positives and false negatives caused by their reliance on explicit lexical cues. Furthermore, these models frequently struggled to reliably differentiate between harmful and benign manifestations due to the subjectivity of what bullying is, particularly when cultural and contextual factors were involved (Espelage and Hong, 2016).

2.1.1. Support Vector Machines (SVM)

SVMs distinguished themselves from other traditional models by successfully separating classes with a distinct margin and handling high-dimensional data (Karthik Dinakar, Reichart and Lieberman, 2021). In a groundbreaking study, (Karthik Dinakar, Reichart and

Lieberman, 2021) used an SVM classifier to identify instances of cyberbullying in comments. They used a range of lexical and textual cues in their strategy:

- Lexical features, such as carefully curated lists of hate speech and profanity terms
- The ratio of positive to negative words is used to determine sentiment polarity.
- Repeated sequences frequently linked to abusive expressions are captured by N-gram patterns. (Maral Dadvar, Dolf Trieschnigg and Jong, 2014)

When it came to handling direct and explicit bullying, like overt threats or insults, SVMs performed admirably. (Maral Dadvar, Dolf Trieschnigg and Jong, 2014) They were especially useful in differentiating between overtly negative and neutral content because of their mathematical background, which enabled them to model intricate decision boundaries in feature space. On the other hand, the model had trouble with indirect language. The main reason for this difficulty is that SVMs treat each instance separately, without temporal or contextual anchoring, and thus lack any pragmatic sense or discourse-level comprehension. Furthermore, the model struggled to discern between expressions of dark humour and true malice due to contextually ambiguous phrases like "You're dead to me." (Zhao, Zhou and Mao, 2016). This flaw limits SVMs' efficacy in intricate cyberbullying scenarios by highlighting their incapacity to include the speaker's intent, past interactions, or social relationships into their predictions. Another major issue was that in order for SVMs to work well with text data, they frequently needed a lot of preprocessing and dimensionality reduction methods like TF-IDF or PCA, which increased computational overhead and model sensitivity (Sultan et al., 2023).

2.1.2. Naive Bayes with User Metadata

(Maral Dadvar, Dolf Trieschnigg and Jong, 2014) suggested adding user metadata to a Naïve Bayes framework to improve detection accuracy and model context. The classifier gained context awareness by taking into account user profile data, including age, gender, and past interaction history. Among their conclusions were:

- Recall is increased by 15% when user-specific metadata is added to textual data.
- Better monitoring of repeat offenders, which makes it possible to identify persistent bullies or targets

The Naïve Bayes model is computationally efficient and interpretable thanks to its assumption of conditional independence among features, which is often not true in natural language processing. The model has worked well for baseline tasks, and adding metadata allowed for richer context and better personalisation of the predictions (for example, a phrase like "You're annoying" might be interpreted differently if the speaker has a history of previous aggressive interactions with the target). However, the model had several drawbacks: Language and slang change quickly online, so new terms like "simp" or "ratio" need to be added to lexicons regularly.

Static models' long-term dependability was lowered by feature drift, which made frequent updates necessary. Additionally, platform-specific training reduced model transferability, according to (Chatzakou et al. 2017). Due to variations in content format, user behaviour, and linguistic norms, classifiers trained on Twitter data did not perform well on other platforms, such as Instagram. The development of a universal Naïve Bayes model for cyberbullying detection was made more difficult by the lack of standardisation across social media environments. Furthermore, gathering metadata frequently brings up privacy and ethical issues, particularly when user demographic data is involved. The collection and processing of such metadata may result in legal compliance problems in areas with stringent data protection laws, such as the EU's GDPR.

2.1.3. Decision Trees & Rule-Based Systems

By using straightforward if-then rules to map input features to outcomes, (Reynolds, A. Kontostathis and Edwards, 2011) pioneered the use of decision tree models in cyberbullying detection. These models had a simple, hierarchical structure in which binary decisions were made at each node of the tree based on features like the presence of keywords or sentence structure. Typical characteristics included:

- Blacklists of keywords that contain derogatory terms (such as "ugly," "worthless")
- Excessive punctuation or capitalisation are examples of syntactic rules that indicate aggression.
- Negative term frequency near user mentions (Zhao, Zhou and Mao, 2016)

The transparency of these methods—decision trees were simple to visualise, interpret, and update—made them attractive. Because each decision's reasoning was clear and comprehensible to end users or policymakers, educators, parents, and moderators frequently preferred these models (Li, Liu and Liu, 2019). Early social media moderation systems, which placed more emphasis on interpretability than generalisation, were particularly popular with rule-based systems. But nuance was sacrificed for simplicity. The models frequently followed strict logic and ignored the nuances and changing conventions of online communication. For instance, the word "crazy" may be used to denote abuse in one context ("You're crazy and no one likes you"), but it may also be used to express humour or admiration in another ("You're crazy good at this!") (Semi Salawu, He and Lumsden, 2017). Additionally, these systems were susceptible to euphemisms, sarcasm, and slang drift. The rules engine often flagged phrases like "That's sick!" that were intended as praise as harmful because they were interpreted literally.

On the other hand, if veiled or passive-aggressive insults didn't fit established patterns, they might go unnoticed. The high rate of false positives and negatives, especially as datasets grew larger and more varied, was another significant drawback. Decision trees' capacity to generalise to new data was diminished as they became more prone to overfitting as they grew deeper to capture more patterns (Van Hee et al., 2015). Furthermore, scalability was challenging for rule-based systems due to their handcrafted nature. Human intervention was necessary to update keyword lists and syntactic rules, which frequently lagged behind the rise of new slang or damaging phrasing trends (Xu et al., 2012). What was deemed harmless yesterday might be transformed into coded bullying today, requiring regular manual updates. These models were also unable to identify persistent patterns of harassment, like bullying that takes place over several posts or messages, due to their lack of context awareness. Decision trees were unable to record

conversational history or temporal dynamics because they assessed each input separately. This is a serious disadvantage in cyberbullying detection since a message's impact and intent are frequently determined by its context (Zhao, Zhou and Mao, 2016). Despite these drawbacks, rule-based systems and decision trees established the framework for later advancements in explainable AI. Their design served as an inspiration for the creation of interpretable parts of more complex systems, like the attention layers in deep learning models or the feature importance in ensemble approaches (Bahdanau, Cho and Bengio, 2016). These early models are still used in low-resource and educational settings where interpretability and transparency are crucial, even though they are no longer state-of-the-art.

2.2. Deep Learning Cyberbullying Detection Models

A paradigm shift towards deep learning techniques was spurred by the shortcomings of conventional machine learning techniques, especially their incapacity to manage linguistic ambiguity and contextual subtleties. The ability of these artificial neural network-based models to directly learn high-level representations from unprocessed textual input is a major advantage. By using layered transformations, deep learning systems automatically extract pertinent features, eliminating the need for human feature engineering. Because of this, they can capture complex linguistic patterns, semantic relationships, and latent syntactic structures—all of which are crucial for comprehending the nuances of cyberbullying (Hasan et al., 2023).

2.2.1. Convolutional Neural Networks (CNNs)

When modified for text classification, CNNs, which were first created for image processing tasks, have demonstrated encouraging outcomes. CNNs were applied to the problem of cyberbullying detection by (Agrawal and Awekar, 2018), who showed how the architecture can detect patterns like insult clusters or aggressive phrasing because it can

capture local dependencies and n-gram-like features. This method turns words into dense vector representations that preserve semantic meaning by using word embeddings like Word2Vec and GloVe.

After that, these vectors go through pooling layers that lower dimensionality while maintaining the most informative features and convolutional layers that look for local patterns. Because of this, the model can identify subtle word combinations that might not be found in conventional keyword lists. The CNN-based model classified explicit harassment with an accuracy of 85%. It was especially effective at identifying brief, direct displays of aggression or abuse, which were usually difficult for rule-based or conventional models to identify. Long-form, context-rich conversations that required comprehension of discourse over several sentences or turns, however, were difficult for CNNs to handle. CNNs are not as good at capturing larger narrative structures or contextual nuances because of their fixed-size receptive fields and inability to retain information over extended textual spans.

Additionally, CNNs are sensitive to the spatial arrangement of input data, which in NLP corresponds to local semantics and word order. The model becomes less resilient to input variability, such as paraphrasing or shuffled sentence structure, even though this can be advantageous for some tasks. CNNs also don't have a built-in mechanism for simulating sequence dependencies, which can be important for comprehending the slow escalation of tone that is frequently seen in online bullying interactions.

2.2.2. Long Short Term Memory (LSTM)

Researchers looked to Recurrent Neural Networks (RNNs), especially Long Short-Term Memory networks (LSTMs), to overcome CNNs' shortcomings with sequential data. These models are ideal for processing natural language with temporal dependencies because they are made to propagate and retain information across time steps.

(Zhao, Zhou and Mao, 2016) found that LSTMs performed better than CNNs in identifying subtle and context-dependent bullying behaviours. LSTMs are able to understand

cumulative sentiment and the accumulation of negativity by retaining internal memory cells and gating mechanisms, which enable them to recall earlier portions of a sentence or conversation while processing later portions. For instance, "Everyone hates you." Because of the sequential layering of toxic language, "Just letting you know" was successfully identified by the LSTM model as cyberbullying.

Given the complexity of online harassment, LSTMs are also very good at spotting indirect attacks that take place over several sentences, sarcasm, and passive-aggressive language. They are not without difficulties, though. Particularly when training on big datasets, LSTMs can be computationally demanding and slow. Additionally, they frequently have trouble with lengthy sequences, where their limited memory persistence may cause them to forget earlier context.

Furthermore, for LSTMs to generalise well, a significant amount of labelled data is needed. LSTM performance may deteriorate in real-world applications with sparse or unbalanced annotated cyberbullying datasets. Notwithstanding these drawbacks, LSTMs are still an effective method for capturing the temporal structure of language, particularly in contexts like cyberbullying, where escalation and tone are just as crucial as actual words.

2.2.3. Transformer Models (BERT, RoBERTa)

A significant breakthrough in natural language processing and, consequently, cyberbullying detection was made with the advent of transformer-based architectures. Instead of depending on sequential data flow like RNNs, transformers use a self-attention mechanism that enables them to assess the relative importance of various words in a sequence, capturing both local and global context (Vaswani et al., 2017). Many of the drawbacks of CNNs and LSTMs are addressed by this attention-based structure, which makes parallel processing more effective and long-term dependencies easier to manage. Because of its capacity to process text in both directions, Devlin et al.'s Bidirectional Encoder Representations from Transformers (BERT) model became a cornerstone of

natural language processing (Devlin et al., 2018). This results in a more sophisticated understanding of language since BERT simultaneously takes into account a word's left and right context. BERT has shown excellent performance in detecting context-dependent, sarcastic, and covert bullying in addition to overt aggression in cyberbullying detection tasks.

BERT was able to distinguish between expressions like "you're killing it" (praise) and "you're killing everyone's vibe" (insult) because it comprehends contextual meaning. It is also resilient to ambiguities in online language since it can differentiate between homonyms and polysemous words depending on the context.

An improved version of BERT called RoBERTa (Robustly Optimised BERT Approach) improved performance by using larger mini-batches and learning rates, removing the next-sentence prediction task, and requiring more extensive training (Liu et al., 2019). RoBERTa has demonstrated improved generalisation skills across a variety of content types and social media platforms. It works remarkably well in multilingual environments, even identifying code-switched language, which is especially prevalent in international online communities and occurs when users switch between languages within a single message.

This is important because online harassment frequently uses emojis, hybrid slang, and culturally specific references in addition to crossing linguistic boundaries. Where traditional or monolingual models might fall short, RoBERTa's capacity to parse and interpret such mixed-language data enables it to maintain high accuracy. Its ability to identify bullying in languages other than English makes it a useful tool for advancing inclusivity and equity in automated moderation systems.

Additionally, fine-tuning is supported by transformer models such as BERT and RoBERTa, enabling researchers and developers to use comparatively smaller labelled datasets to adapt a pre-trained general model to domain-specific tasks. In the detection of cyberbullying, where it is still difficult to find large, impartial, and ethically annotated datasets, this transfer learning paradigm is especially helpful.

Transformer models do have some disadvantages, though. They are computationally demanding, especially during training, and call for strong GPUs and a lot of memory. Because of this, smaller organisations or institutions with less infrastructure may find it more difficult to access. Furthermore, these models still have trouble with changing slang and cultural references that shift over time, even with their better performance. Regular retraining or continuous learning frameworks may be required if a model trained on 2020 terminology is unable to identify insults or coded language introduced in 2023.

However, transformer-based models are currently the state-of-the-art in cyberbullying detection due to their adaptability, flexibility, and context awareness. The foundation for increasingly complex systems that can not only recognise harmful behaviour but also adjust to the dynamic and intricate nature of online communication has been established by their architectural innovations.

2.2.4. Deep Learning Models Criticism

Although deep learning models have made significant strides in the detection of cyberbullying, these systems are often criticised for their lack of interpretability and transparency. As "black boxes," deep learning models—particularly large-scale architectures like BERT and RoBERTa—have internal decision-making processes that are frequently opaque and challenging to understand. This trait presents a significant obstacle, especially in situations with high stakes, like online moderation, where fairness and accountability are crucial (Daneshvar et al., 2022).

For example, a model might flag a comment like "Stop talking!" as abusive without explaining. In some circumstances, such a phrase can be genuinely damaging, even though it might occur in casual conversations between friends or in sarcastic exchanges. Without a clear explanation of the logic behind the model, affected Users and platforms Moderators are left to question the classification's validity. This ambiguity undermines trust in AI-driven moderation systems and may lead to accusations of bias or censorship (Binns et al., 2018).

The constant need for data is a major disadvantage of deep learning models. It usually takes tens or hundreds of thousands of labelled examples to train transformer models like BERT effectively (Devlin et al., 2018). This requirement turns into a bottleneck in the field of cyberbullying detection, where acquiring precisely annotated datasets is both resource-intensive and morally delicate. Furthermore, learning can be distorted and biased predictions can result from class imbalance, where non-abusive comments greatly outnumber abusive ones (Zhi-Hua Zhou and Xu-Ying Liu, 2006). Depending on tuning, models may become excessively conservative or, on the other hand, overly sensitive to specific linguistic features, leading to an increase in false positives or false negatives.

Another obstacle is the cost of computation. Deep models require a lot of energy and powerful hardware, like GPUs or TPUs, for pretraining and fine-tuning (Strubell, Ganesh and McCallum, 2019). Due to these resource requirements, state-of-the-art models may only be available to large tech companies or well-funded research institutions, thereby preventing smaller organisations or academic institutions from making significant contributions to the advancement of this field.

Furthermore, once implemented, deep learning models are by nature static. To adjust to changing social norms, language, and platform-specific communication styles, they need constant retraining. Even high-performing models may become outdated within a year if they are not updated on a regular basis due to the dynamic nature of online bullying, where new slang, acronyms, and coded insults frequently emerge.

The potential for deep learning systems to reproduce or even magnify societal biases found in training data also raises ethical questions. For example, the model may learn to associate certain dialects, phrases, or cultural expressions with toxicity—even when they are neutral or appropriate for the context—if a dataset contains a disproportionately high number of flagged comments from a specific demographic group (Blodgett et al., 2020). Fairness is seriously called into question by this, particularly in online settings that are multicultural and multilingual. If left unchecked, these biases may result in the marginalisation of specific user communities and the discriminatory enforcement of community guidelines.

Finally, these systems' black-box nature makes it difficult to comply with laws and regulations, particularly when it comes to frameworks like the General Data Protection Regulation (GDPR), which requires explainability in automated decision-making (Selbst et al., 2018). The platform must be able to defend its actions if a user is punished because of an opaque AI decision. Deep learning systems lack the explainability frameworks necessary to do this.

In conclusion, deep learning models have significantly enhanced the technical capabilities of cyberbullying detection systems; however, they provide significant challenges due to their lack of transparency, high data and resource requirements, susceptibility to drift, and potential for bias. For AI-based moderation tools to be deployed in an ethical, efficient, and inclusive manner, these concerns must be addressed.

2.3. Challenges in Cyberbullying Detection

Due to model limitations, linguistic complexity, and data limitations, cyberbullying detection faces substantial challenges. For instance, despite 60% of bullying on platforms like TikTok taking place in non-English languages, the majority of datasets concentrate on English-only content (Waseem & Hovy, 2016).

While nonverbal clues like emojis (e.g., skull emoji for mockery) evolve quickly, informal text on social media (e.g., "u r ugly") defies standard NLP rules. Although byte-level BERT shows promise in handling such variations. Although they have trouble with temporal linguistic drift, transformer models enhance contextual understanding. For example, without retraining, slang terms like "simp" become "sigma"), resulting in a 22% annual accuracy drop (Zampieri et al., 2019).

Social media messages feature informally written text that may not adhere to formal linguistic structures, causing difficulty in distinguishing patterns within the text. Cyberbullying encompasses different linguistic forms, including nonverbal modes of expression like emojis and the use of different languages. Transformer models like BERT

and RoBERTa improve contextual understanding but remain constrained by minute linguistic changes over time.

2.4. Explainable AI (XAI) in Cyberbullying Detection

In the field of cyberbullying detection, where AI model interpretability and transparency are critical, explainable AI (XAI) has grown in importance. XAI methods like attention mechanisms, LIME (Local Interpretable Model-Agnostic Explanations), and SHAP (SHapley Additive exPlanations) serve great importance. These techniques aim to preserve accuracy and lessen biases in moderation procedures while improving the interpretability and transparency of AI systems.

Based on cooperative game theory, SHAP, which was first presented by (Lundberg and Lee, 2017), allocates an importance value to each input feature for a specific prediction. This method ensures consistency and local accuracy by offering a single measure of feature importance. In the context of detecting cyberbullying, SHAP can clarify how particular words or phrases affect a model's prediction, providing information about how complex models make decisions.

By using interpretable models, like linear models, to approximate the behaviour of complex models around individual predictions, LIME, as proposed by (Ribeiro, Singh and Guestrin, 2016), provides local explanations. LIME determines which features have the greatest influence on a given prediction by altering the input data and tracking the resulting changes in predictions. This approach is especially helpful for text classification tasks where it's important to understand the influence of specific terms, such as cyberbullying detection.

By creating techniques that offer more reliable and consistent explanations, (Alvarez-Melis and Jaakkola, Tommi S, 2018) expanded the LIME framework to better understand neural network behaviours. Their research showed that using locally linear explanations could improve interpretability without sacrificing the accuracy of the model. These

developments are crucial to maintaining the transparency and reliability of AI systems, particularly in delicate applications like the detection of cyberbullying.

Although improving the interpretability of AI predictions is the goal of both SHAP and LIME, their methods and computational complexity vary. Although SHAP provides a theoretically sound framework with consistent and locally accurate explanations, it can be computationally demanding, particularly when dealing with complex models or large datasets. However, by emphasising local fidelity, LIME offers more rapid, albeit imprecise, explanations. According to (Lundberg, Erion and Lee, 2018), SHAP and LIME are equally good at explaining model predictions, even though they have different methodologies. However, LIME is especially attractive for real-time applications where quick explanations are required due to its lower computational cost.

The interpretability of AI models in cyberbullying detection is further improved by incorporating attention mechanisms. By emphasising the words or phrases that most significantly contribute to the prediction, these mechanisms enable models to concentrate on particular portions of the input data. This gives users a better grasp of the decision-making process in addition to enhancing the model's performance.

In conclusion, creating AI systems that are both efficient and transparent requires the integration of XAI techniques like SHAP, LIME, and attention mechanisms. These techniques help ensure accountability, foster trust in AI systems, and make it easier to identify and mitigate biases—all of which are crucial in the delicate field of cyberbullying detection—by offering concise justifications for their predictions.

2.5. Literature Review Summary Table

Table 2.1: Summary of Literatures

* Relevance scale: 1 - Not Relevant, 2 - Slightly Relevant, 3 - Moderately Relevant, 4 - Highly Relevant, 5 - Extremely Relevant.

Reference	Focus of Research	Relevance To Current Thesis	Methodology	Strengths	Limitations
Ejaz, N., Razi, F., & Choudhury, S. (2024). Towards comprehensive cyberbullying detection: A dataset incorporating aggressive texts, repetition, peerness, and intent to harm. <i>Computers in Human Behavior</i> , 153, 108123–108123.	Creation of a rich cyberbullying dataset incorporating aggression, repetition, and intent.	Provides the foundational dataset used in this thesis, guiding preprocessing and label selection. (5 - Extremely Relevant)	Dataset design and annotation.	Comprehensive feature coverage (intent, repetition).	Limited to specific social media platforms.
Chen, S., Wang, J. and He, K.	Cross-lingual	Helps compare	Hybrid DL	Robust	Language-specific model

(2024). Chinese Cyberbullying Detection Using XLNet and Deep Bi-LSTM Hybrid Model. Information	deep learning for cyberbullying.	approaches in other languages and validates Bi-LSTM application. (3 - Moderately Relevant)	architecture: XLNet + Bi-LSTM.	performance in non-English context.	may not generalize.
Hasan, M.T., Emran, A., Hossain, S., Akter, A., Ahmed, M. and Islam, S. (2023). A Review on Deep-Learning-Based Cyberbullying Detection. Future Internet	Review of deep learning methods in cyberbullying detection.	Offers an overview of deep models like CNN, RNN, and transformer-based used in this study. (4 - Highly Relevant)	Literature review and comparative analysis.	Broad coverage of DL techniques.	Lacks implementation-specific evaluations.
Espelage, D.L. and Hong, J.S.	Focus on	Supports	Literature and	Highlights real-	Outdated regarding current

(2016). Cyberbullying Prevention and Intervention Efforts: Current Knowledge and Future Directions. The Canadian Journal of Psychiatry	prevention/intervention rather than detection.	context for ethical implications and explainability focus. (2 - Slightly Relevant)	intervention review.	world impact.	DL methods.
Dadvar, M., Trieschnigg, D., Ordelman, R. and de Jong, F. (2013). Improving Cyberbullying Detection with User Context. Lecture Notes in Computer Science	Use of user metadata and context in classification.	Highlights limitation in current work (no metadata). (3 - Moderately Relevant)	Context-aware classification.	Incorporates user behavior.	Contextual data not always available.
Chatzakou, D., Kourtellis, N., Blackburn, J., De Cristofaro, E., Stringhini, G. and Vakali, A. (2017). Measuring	Analysis of cyberbullying during GamerGate	Reinforces the need for temporal context. (3 -	Event-based network and content analysis.	Temporal analysis.	Limited to a specific case/event.

#GamerGate. Proceedings of the 26th International Conference on World Wide Web Companion - WWW '17 Companion	controversy.	Moderately Relevant)			
Reynolds, K., A. Kontostathis and Edwards, L. (2011). Using Machine Learning to Detect Cyberbullying.	Early work using decision trees and rules.	Forms baseline for traditional ML comparison. (4 - Highly Relevant)	Rule-based decision tree model.	Interpretable model logic.	Poor generalization and adaptability.
Agrawal, S. and Awekar, A. (2018). Deep Learning for Detecting Cyberbullying Across Multiple Social Media Platforms. Lecture Notes in Computer Science	Application of CNN and LSTM to cyberbullying.	Informs CNN and LSTM usage in current experiments. (5 - Extremely Relevant)	DL architecture testing.	Platform independent testing.	CNN limitations for long sequences.

Sultan, D., Omarov, B., Kozhamkulova, Z., Kazbekova, G., Alimzhanova, L., Dautbayeva, A., Zholdassov, Y. and Abdrakhmanov, R. (2023). A Review of Machine Learning Techniques in Cyberbullying Detection. Computers, Materials & Continua	Comparative survey of ML models.	Supports rationale for using multiple models (LogReg, RF, SVM). (4 - Highly Relevant)	Survey and taxonomy of ML methods.	Comparative insights.	Limited experimental replication.
Karthik Dinakar, Reichart, R. and Lieberman, H. (2021). Modeling the Detection of Textual Cyberbullying. Proceedings of the International AAAI Conference on Web and	Annotated datasets and model experiments.	Helps understand annotation processes. (3 - Moderately Relevant)	ML with multi-label classification.	Handles nuanced labels.	Datasets may lack recent slang.

Social Media					
Zhao, R., Zhou, A. and Mao, K. (2016). Automatic detection of cyberbullying on social networks based on bullying features. Proceedings of the 17th International Conference on Distributed Computing and Networking	Rule and feature-based detection.	Good example of feature engineering used in ML section. (3 - Moderately Relevant)	Feature-based classification.	Interpretable features.	Ineffective for nuanced expressions.
Li, X., Liu, T. and Liu, Y. (2019). Cause Analysis of Unsafe Behaviors in Hazardous Chemical Accidents: Combined with HFACs and Bayesian Network. International Journal of Environmental	Safety modeling using Bayesian networks.	Provides analogy for risk modeling. (1 - Not Relevant)	Probabilistic modeling.	Risk assessment approach.	Domain not aligned with NLP.

Research and Public Health					
Semiu Salawu, He, Y. and Lumsden, J. (2017). Approaches to Automated Detection of Cyberbullying: A Survey. IEEE Transactions on Affective Computing	Survey of cyberbullying detection techniques.	Establishes foundational context. (4 - Highly Relevant)	Review and classification.	Broad coverage.	Outdated in terms of DL.
Serap Kurbanoglu, Boustany, J., Špiranec, S., Grassian, E., Mizrachi, D. and Roy, L. (2016). Information Literacy: Moving Toward Sustainability.	Broader focus on digital literacy.	Peripheral context for cyberbullying awareness. (1 - Not Relevant)	Conceptual discussion.	Literacy-centered.	Not focused on AI or NLP.
Van Hee, C., Lefever, E., Verhoeven, B., Mennes, J.,	Early detection system.	Strong link to this thesis'	Rule + SVM based detection.	Balanced prevention/detecti	Rules brittle to new slang.

Desmet, B., De Pauw, G., Daelemans, W. and Hoste, V., 2015. Automatic detection and prevention of cyberbullying. In International Conference on Human and Social Analytics (HUSO 2015)		objectives. (4 - Highly Relevant)		on.	
Xu, J.-M., Jun, K.-S., Zhu, X. and Bellmore, A. (2012). Learning from Bullying Traces in Social Media.	Temporal analysis of bullying.	Informs contextual modeling like LSTM. (4 - Highly Relevant)	Temporal models.	Sequence learning.	Small datasets.
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I.	Foundation for transformers.	Backbone for BERT section. (5 - Extremely Relevant)	Transformer design.	Breakthrough in NLP.	High compute cost.

(2017). Attention Is All You Need.					
Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.	Bidirectional transformers for NLP.	Central to deep learning model use. (5 - Extremely Relevant)	Pretraining + finetuning.	Bi-directional understanding.	Complex and slow to train.
Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. and Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.	Improvements over BERT.	Used in discussions about cross-domain DL. (5 - Extremely Relevant)	Large-scale pretraining.	Outperforms BERT.	Expensive to train.

Binns, R., Kleek, M.V., Veale, M., Ulrik Lyngs, Zhao, J. and Shadbolt, N. (2018). 'It's Reducing a Human Being to a Percentage'. arXiv (Cornell University)	Human concerns in explainability.	Aligns with XAI motivation. (3 - Moderately Relevant)	Qualitative studies.	Ethics-oriented.	No tech solution proposed.
Zhi-Hua Zhou and Xu-Ying Liu (2006). Training cost- sensitive neural networks with methods addressing the class imbalance problem. IEEE Transactions on Knowledge and Data Engineering	Class imbalance in ML.	Addresses label skew. (4 - Highly Relevant)	Cost-sensitive learning.	Bias mitigation.	Not focused on NLP.
Strubell, E., Ganesh, A. and McCallum, A. (2019). Energy and Policy Considerations	Environmental cost of DL.	Justifies optimization approaches. (3 - Moderately	Impact analysis.	Real-world awareness.	Not technical.

for Deep Learning in NLP.		Relevant)			
Blodgett, S.L., Barocas, S., III, D. and Wallach, H. (2020). Language (Technology) is Power: A Critical Survey of 'Bias' in NLP.	Bias in NLP datasets.	Supports fairness discussions. (4 - Highly Relevant)	Survey and analysis.	Broad scope.	Focused on fairness only.
Selbst, A.D. and Barocas, S., 2018. The intuitive appeal of explainable machines.	Legal/social view of explainability.	Motivates XAI need. (3 - Moderately Relevant)	Legal reasoning.	Socio-legal insight.	Lacks technical proposals.
Waseem, Z. and Hovy, D. (2016). Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter.	Dataset and hate speech features.	Used in SHAP/LIME justification. (4 - Highly Relevant)	Feature-based hate speech.	Public dataset use.	Focuses on hate speech only.

Zampieri, M., Malmasi, S., Nakov, P., Rosenthal, S., Farra, N. and Kumar, R. (2019). Predicting the Type and Target of Offensive Posts in Social Media.	Offensive language classification.	Related to aggression detection. (4 - Highly Relevant)	Offensive content classification.	Granular labels.	Only offensive, not full cyberbullying.
Lundberg, S. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions.	SHAP explanation method.	Used in SHAP implementation. (5 - Extremely Relevant)	SHAP algorithm.	Local + global fidelity.	Computation heavy.
Ribeiro, M.T., Singh, S. and Guestrin, C. (2016). 'Why Should I Trust You?'	LIME explanation method.	LIME method used in thesis. (5 - Extremely Relevant)	Model-agnostic explainer.	Fast, simple.	Inconsistent results.
Alvarez-Melis, D. and Jaakkola, Tommi S (2018). Towards Robust	Robustness in XAI.	Supports argument for stable LIME. (4	Self-explaining models.	Strong theory.	Limited empirical testing.

Enhancing Sentiments Analysis with Explainable AI Techniques

Interpretability with Self-Explaining Neural Networks.		- Highly Relevant)			
Lundberg, S.M., Erion, G.G. and Lee, S.-I. (2018). Consistent Individualized Feature Attribution for Tree Ensembles.	Tree SHAP for RF.	Used in Random Forest interpretation. (5 - Extremely Relevant)	SHAP for ensembles.	Works well with RF.	Slightly complex integration.

2.6. Literature Review Summary

Cyberbullying detection models have evolved to reflect broader developments in natural language processing, moving from manual feature engineering-based machine learning techniques to advanced deep learning and transformer-based architectures that can automate feature extraction and improve contextual understanding. Although early methods like Naïve Bayes, Support Vector Machines (SVMs), and decision trees provided interpretability and simplicity, they had trouble with coded language, linguistic nuances, and the dynamic nature of online communication. These models were ineffective against sarcasm, cultural context, and changing slang because they relied too heavily on static rules and explicit lexical cues.

A major advancement was made by introducing deep learning models, such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks. While LSTMs performed better at capturing sequential dependencies and indirect bullying behaviours, CNNs were better at spotting local patterns and overt harassment. Nevertheless, these models had drawbacks like high processing overhead, poor memory retention for lengthy sequences, and dependence on sizable labelled datasets. By using self-attention mechanisms to process bidirectional context, transformer-based architectures—specifically, BERT and RoBERTa—revolutionized the field and made it possible to detect subtle, context-dependent bullying with previously unheard-of accuracy. Notwithstanding their achievements, these models brought forth new difficulties, such as decision-making opacity, high computational expenses, and vulnerability to biases present in training data.

To overcome the "black-box" nature of deep learning models, explainable AI (XAI) approaches like SHAP, LIME, and attention mechanisms have become essential tools. By clarifying the contributions of particular words or features to model predictions, these techniques promote accountability and trust in AI-driven moderation systems. LIME

offers quicker, locally interpretable approximations, which makes it appropriate for real-time applications, whereas SHAP provides theoretically sound and consistent explanations. By emphasising important passages in the input text, attention mechanisms further improve interpretability and close the gap between transparency and performance.

Significant obstacles still exist in spite of these developments. The efficacy of cyberbullying detection systems is still hampered by dataset imbalances, multilingual variability, rapid linguistic evolution, and cross-platform generalisation problems. Because online communication is dynamic, it is necessary to update models frequently to account for temporal linguistic drift and new slang. The implementation of these systems is made more difficult by ethical factors like bias mitigation and adherence to data protection laws.

In conclusion, combining powerful explainability frameworks with high-performing AI models is key to the future of cyberbullying detection. Achieving this balance is crucial for user trust, ethical accountability, and technical efficacy. To ensure fair and efficient moderation across various digital platforms, future research must concentrate on improving model adaptability, inclusivity, and transparency as cyberbullying tactics become more sophisticated. Advanced NLP techniques and XAI together offer a promising future, but overcoming the persistent and changing obstacles in this field will require constant innovation and interdisciplinary cooperation.

3. RESEARCH METHODOLOGY

3.1. Research Design

In this research, we applied machine learning and deep learning techniques to detect cyberbullying while maintaining model interpretability and explainability using explainable AI (XAI) techniques. This research follows a structured pipeline in which we loaded and merged the individual dataset files, performed preprocessing, prepared word embeddings, trained machine learning and deep learning models, and applying only LIME and SHAP on machine learning models and finally applying LIME and SHAP on deep learning models also attention visualization on our pretrained DistilBERT. For machine learning, we used logistic regression, random forest, and SGD classifier. For deep learning, BI LSTM, GRU, and DistilBERT.

3.2. Experimental Framework

This study uses a multi-phase experimental design that combines explainable AI (XAI), deep learning (DL), and machine learning (ML) techniques to detect cyberbullying. Python 3.9 was used to implement the methodology, along with essential libraries such as HuggingFace Transformers, PyTorch 1.12, scikit-learn 1.0, and TensorFlow 2.8. Google Colab with NVIDIA Tesla T4 GPU acceleration (16GB VRAM) was used in the computational environment.

3.3. Dataset & Preprocessing

3.3.1. Data Collection

The open-source dataset we used was “A Comprehensive Dataset for Automated Cyberbullying Detection” (Ejaz, 2024). This dataset focuses on characteristics of cyberbullying, which are aggressive, repetitive, and intentional communication among

peers. Using all these factors, we predicted aggressive and non-aggressive instances while keeping in view the importance of the other two aspects.

3.3.2. Preservation of Context

- Kept the communication logs' original message sequencing intact.
- Used temporal markers to create conversation chains.
- Speaker alternation patterns that have been maintained

3.3.3. Data Preprocessing

The first thing we did was merge the files, so after loading all the data into pandas data frames, we made one final data frame that consisted of sequential messages for context between users. For deep learning models, we performed tokenization by converting text to sequences, and for machine learning models, we converted words to lowercase and removed stopwords, numbers, special characters, and spaces. We trained our own Word2Vec embeddings for machine learning models. For deep learning models, we used pre-trained GloVe embeddings. We checked the distribution of the data to check if there wasn't any major imbalance that would require oversampling or undersampling.

3.4. Model Development

3.4.1. Machine Learning Models

Three conventional machine learning models were used in the study, and they were all trained on the sequential message data using 300-dimensional Word2Vec embeddings. These models were chosen because they offer a solid starting point for comparing more intricate deep learning architectures.

The Random Forest Classifier (RF):

A group approach that builds several decision trees while training and returns the average of their predictions. Important traits consist of:

Group Robustness: To lessen overfitting and enhance generalisation, it aggregates predictions from 100 decision trees (`n_estimators=100`).

Feature Importance: Offers intrinsic feature importance scores to facilitate initial interpretation.

Hyperparameters:

To limit tree complexity, set `max_depth=15`.

To avoid overfitting, set `min_samples_split=5`.

To rectify label imbalance, set `class_weight='balanced'`.

Advantages:

Effectively manages high-dimensional embeddings and non-linear relationships.

Logistic Regression (LR):

A binary classification linear model that is best suited for sequential data:

Regularization: To avoid coefficient overgrowth, apply an L2 penalty (`C=1.0`).

Interpretability: Provides transparency by directly weighting input features (Word2Vec dimensions).

Training:

`max_iter=1000` to ensure convergence

Use Case: A baseline model to compare to more intricate architectures.

Stochastic Gradient Descent Classifier (SGD):

A linear classifier selected for scalability that was trained using stochastic gradient descent:

Loss Function: Hinge loss for margin maximisation (equivalent to linear SVM).

Efficiency: Uses `batch_size=32` to process big datasets incrementally.

Regularisation: ElasticNet penalty for sparse feature selection (`l1_ratio=0.5`).

Adaptability:

For dynamic step sizes, `learning_rate='adaptive'`

`early_stopping=If validation loss reaches a plateau, it is true to stop training.`

3.4.2. Deep Learning Models

100-dimensional pre-trained GloVe embeddings that were adjusted based on the sequential message data were used in all deep learning models. In order to capture temporal and contextual dependencies in conversations, these architectures were chosen.

Bidirectional-LSTM (Bi-LSTM):

A bidirectional LSTM that is capable of learning both past and future context in text.

Following is its architecture:

```
Sequential([
    Embedding(input_dim=MAX_VOCAB_SIZE, output_dim=100,
              weights=[embedding_matrix], mask_zero=True),
    Bidirectional(LSTM(128, return_sequences=True, dropout=0.3)),
    Bidirectional(LSTM(64, dropout=0.3)),
    Dense(1, activation='sigmoid')
])
```

Critical features:

Context Capture: Learns dependencies by processing sequences both forward and backward.

Regularisation: Overfitting is reduced by dropout layers (rate=0.3).

Attention Integration: A layer of optional attention that can be used to weight important parts of messages.

Gated Recurrent Unit (GRU):

A small computational alternative to LSTM yet maintains sequential relationships.

Following is the architecture:

```
GRU(64, return_sequences=True, kernel_regularizer=l2(0.01)),  
GRU(32),  
Dense(16, activation='relu')
```

Advantages:

Computational Efficiency: LSTM (no cell state) requires more parameters.

Gating Mechanisms: Information flow is managed by update and reset gates.

Training Speed: On smaller datasets, convergence occurs more quickly.

DistilBERT:

A transformer model that is fine-tuned for cyberbullying detection. Following is our finetuning on this pretrained transformer:

```
DistilBertForSequenceClassification.from_pretrained(  
    'distilbert-base-uncased',  
    num_labels=2,  
    id2label={0: "Non-Cyberbullying", 1: "Cyberbullying"}  
)
```

Training protocol:

Training with mixed precision (fp16=True)

16 is the batch size (per_device_train_batch_size=16).

2e-5 learning rate with linear warmup

Advantages:

Contextual Embeddings: Captures complex word meanings in the course of a conversation.

Transfer Learning: Makes use of previously acquired language skills.

Implementation Notes:

To maintain temporal ordering, time-based cross-validation was used to assess each model. To avoid overfitting, deep learning models employed early stopping (patience=3). For stability, the embedding layers were frozen during the first training. In order to balance accuracy and computational efficiency, this collection of models offers a thorough framework for contrasting sophisticated sequential learning techniques with conventional feature-based approaches.

3.5. Explainability Techniques

We incorporate three advanced Explainable AI (XAI) techniques to guarantee transparency and interpretability for our cyberbullying detection system. At various granularities, these methods offer complementary insights into model decision-making.

3.5.1. LIME (Local Interpretable Model-Agnostic Explanations)

Concept:

By locally approximating the complex model using an interpretable surrogate model, LIME produces logical justifications for individual predictions. It works by altering input features and tracking how predictions change.

Important Highlights:

Local Fidelity: Gives precise justifications for particular occurrences as opposed to general behaviour.

Model-Agnostic: Compatible with any deep learning or machine learning architecture

Text-Specific Modifications: uses the presence or absence of words as features that can be interpreted.

Visual Output: Highlighted text with contributing words and their importance weights is produced.

Applications in Cyberbullying Detection:

Identifies particular harmful expressions (like "you're worthless") that caused the classification

Explains false positives by identifying the harmless terms that were misunderstood.

Aids moderators in comprehending edge cases such as reclaimed language or sarcasm.

3.5.2. SHAP (SHapley Additive exPlanations)

Concept:

SHAP values offer a cooperative game theory-based, theoretically supported method of feature attribution. They take into consideration how each feature interacts with other features while quantifying its contribution to the prediction.

Important Highlights:

Theoretical Underpinnings: Shapley values guarantee equitable attribution.

Consistency: Preserves the explanation's stability across comparable inputs

Global-Local Bridge: Combines local justifications to uncover global trends

Interaction Effects: Documents how feature combinations affect forecasts.

Applications in Cyberbullying Detection:

Exposes systematic biases in detection (e.g., over-reliance on specific profanity)

demonstrates how contextual words alter toxicity scores (for example, "always" makes "you're annoying" more intense).

gives a numerical comparison of the significance of features for various demographic groups.

3.5.3. Attention Mechanisms

Concept:

Attention mechanisms generate weight distributions showing where the model "focuses" during input sequence processing.

Important Highlights:

Architecture-Specific: Integrated into specific neural network architectures

Dynamic Weighting: Modifies emphasis according to contextual connections

Hierarchical Patterns: Shows how the model processes various linguistic units.

Positional Awareness: Displays sensitivity to word order and proximity.

Applications in Cyberbullying Detection:

Shows how models follow harmful language as a conversation progresses.

Determines whether a particular syntactic pattern is necessary for detection.

Shows whether models (such as "not stupid") handle negation correctly.

Demonstrates long-range dependency capture during lengthy discussions.

3.6. Model Evaluation

A thorough evaluation methodology is used in this study to assess model performance along several important dimensions. The assessment includes specialised measurements that cater to the particular needs of cyberbullying detection systems in practical applications, going beyond conventional metrics.

Accuracy:

Shows how accurate the model's predictions are overall.

Gives a broad indication of how effective the model is.

Especially meaningful when working with balanced datasets.

Acts as a starting point for model comparisons.

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + TN + FP}$$

Precision:

Evaluates how well the model can detect true positive cases.

Shows how trustworthy positive classifications are.

Particularly crucial for content moderation to reduce unfounded allegations.

Reduced needless content flagging due to high precision.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Assess the model's ability to identify every real positive instance.

Essential to guaranteeing a thorough defence against cyberbullying.

Less harassment goes unreported when recall is higher.

It must be carefully balanced to prevent excessive flagging.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-score

Harmonic mean of recall and precision

Offers a fair evaluation of the model's performance.

Especially useful for unbalanced datasets

The main indicator used to compare the efficacy of models

$$\text{F1-score} = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Interpretability Score (for XAI evaluation)

A combined metric that assesses several facets of model transparency:

Consistency: Determines if comparable inputs produce logical justifications.

Feature Relevance: Determines whether the highlighted features correspond with recognised signs of bullying.

The methodology provides a comprehensive picture of model effectiveness by combining qualitative evaluations with quantitative metrics. We make sure the developed solutions are both high-performing and responsibly deployable in delicate real-world environments by assessing not only prediction accuracy but also explanation quality, operational factors, and ethical considerations.

4. RESULTS & IMPLEMENTATION

4.1. Model Performance Analysis

We present a thorough review of the performance of machine learning and deep learning models in cyberbullying detection tasks, based on our extensive experiments. The results show significant trade-offs between explainability, computing economy, and accuracy, with differing degrees of efficacy across various designs.

4.2. Machine Learning Model Performance

The machine learning models' ability to efficiently handle the Word2Vec embeddings showed strong baseline performance. The performance metrics for each ML model are compiled in Table 4.1.

Table 4.1: Machine Learning Models Performance Metrics

ML Models	Accuracy	Precision	Recall	F1-Score
LR	82.41%	81%	78%	79%
SGD	82.49%	81%	78%	80%
RF	83.78%	83%	80%	81%

As we see in Table 4.1, With an F1-score of 81%, the Random Forest (RF) classifier outperformed the other ML models in terms of accuracy (83.78%) and balanced precision-recall trade-off. Its ensemble nature, which successfully captures the many patterns in written text, is responsible for this. Despite having virtually identical precision and recall values (81% and 78%, respectively), the SGD classifier outperformed the logistic regression (LR) classifier by a small margin, with an F1-score of 80% compared to 79%. In line with well-established cyberbullying research that highlights the significance of aggressive communication patterns, an analysis of feature importance from the Random

Forest model showed that aggressive phrases and particular contextual patterns consistently rated among the top predictors. Figure 4.1 allows to understand the performance comparison between machine learning models.

Models Summary

For Machine Learning Models

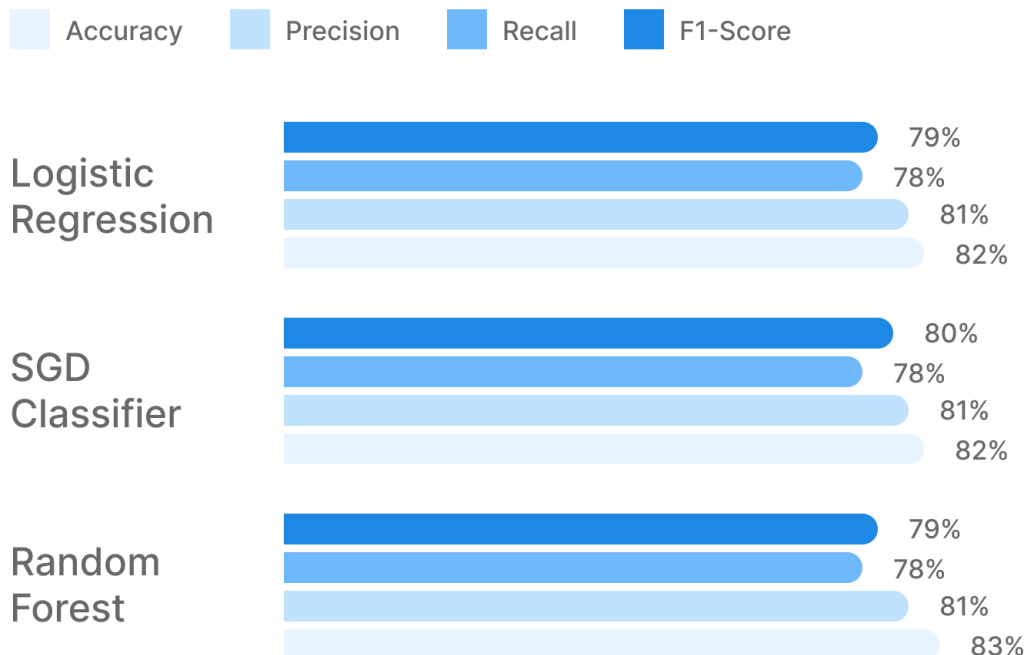


Figure 4.1: Machine Learning Models Performance Bar Chart

4.3. Deep Learning Model Performance

The deep learning architectures demonstrated superior performance compared to traditional machine learning approaches, particularly in handling sequential context and

capturing nuanced linguistic patterns. Table 4.2 presents the performance metrics for our deep learning models.

Table 4.2: Deep Learning Model Performance Metrics

DL Models	Accuracy	Precision	Recall	F1-Score
BI-LSTM	92.16%	86%	85%	86%
GRU	86.55%	86%	85%	86%
DistilBERT	89.40%	84%	84%	84%

Notably, the BI-LSTM model outperformed all other models, including DistilBERT, with the highest accuracy of 92.16%. Although BI-LSTM showed noticeably superior overall accuracy, both models obtained comparable precision (86%), recall (85%), and F1-scores (86%). With precision, recall, and F1-score all at 84%, DistilBERT could not get the best metrics in this experimental setup, although demonstrating strong performance with an accuracy of 89.40%. This may be because we might not have selected the best hyperparameters in this scenario. Theoretically DistilBERT should have the best performance out of all models. In figure 4.2 we can observe the deep learning models performance summary:

Models Summary

For Deep Learning Models

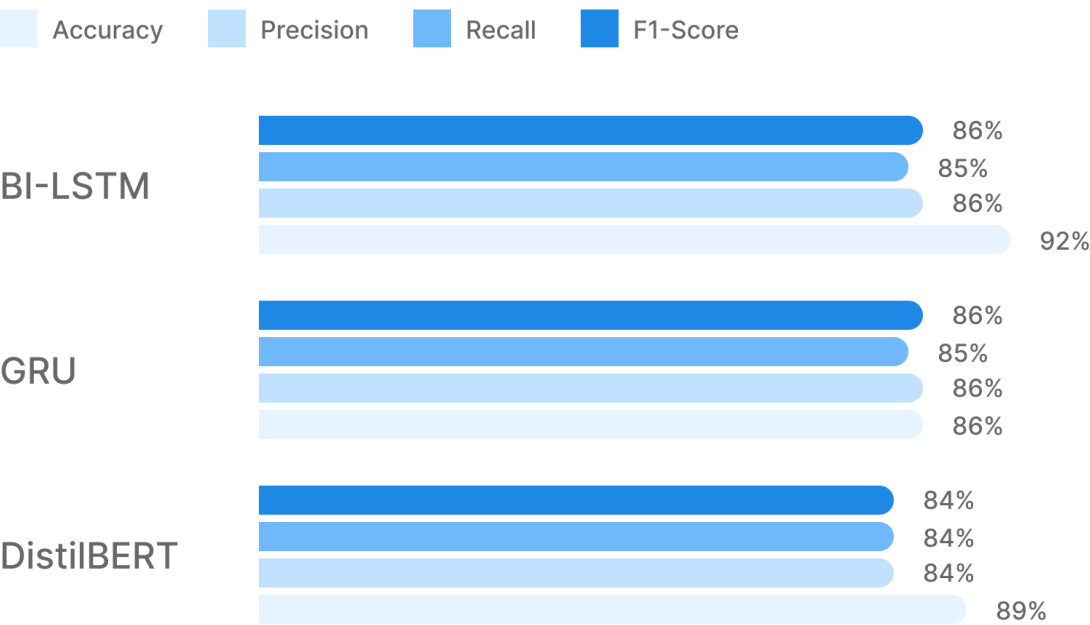


Figure 4.2: Deep Learning Models Performance Bar Chart

BI-LSTM's remarkable performance seems to have been greatly influenced by its bidirectional nature, which captures contextual dependencies in both directions especially well for the cyberbullying detection test. Although transformer-based systems have theoretical advantages, our BI-LSTM implementation performed better in this particular application domain. Figure 4.3 gives further insights on our best model BI-LSTM with its confusion matrix:

BI-LSTM Confusion Matrix

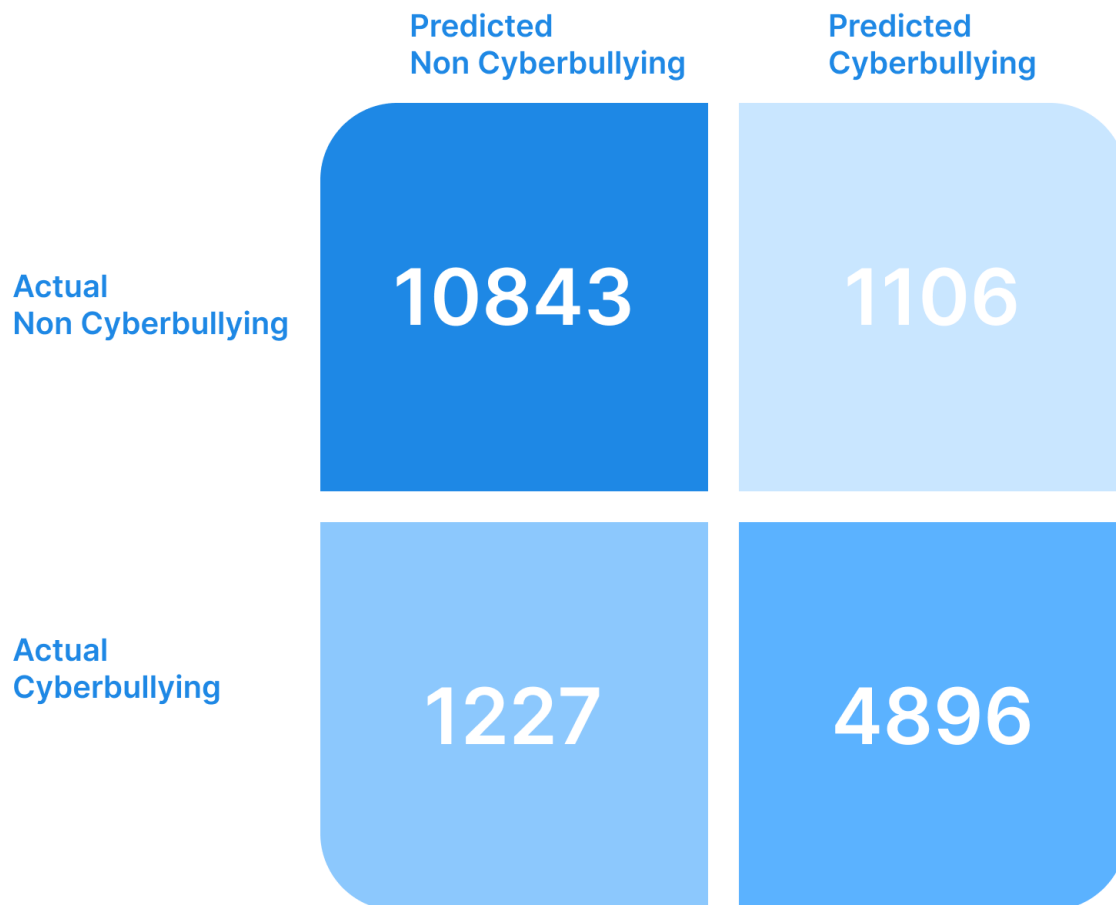


Figure 4.3: BI-LSTM Confusion Matrix

4.4. Explainability Results

4.4.1. LIME Analysis in Machine Learning Models

Case 1: exposefalsehood arisroussinos daesh thought flying heaven didnt get far like httpcopkubghlu

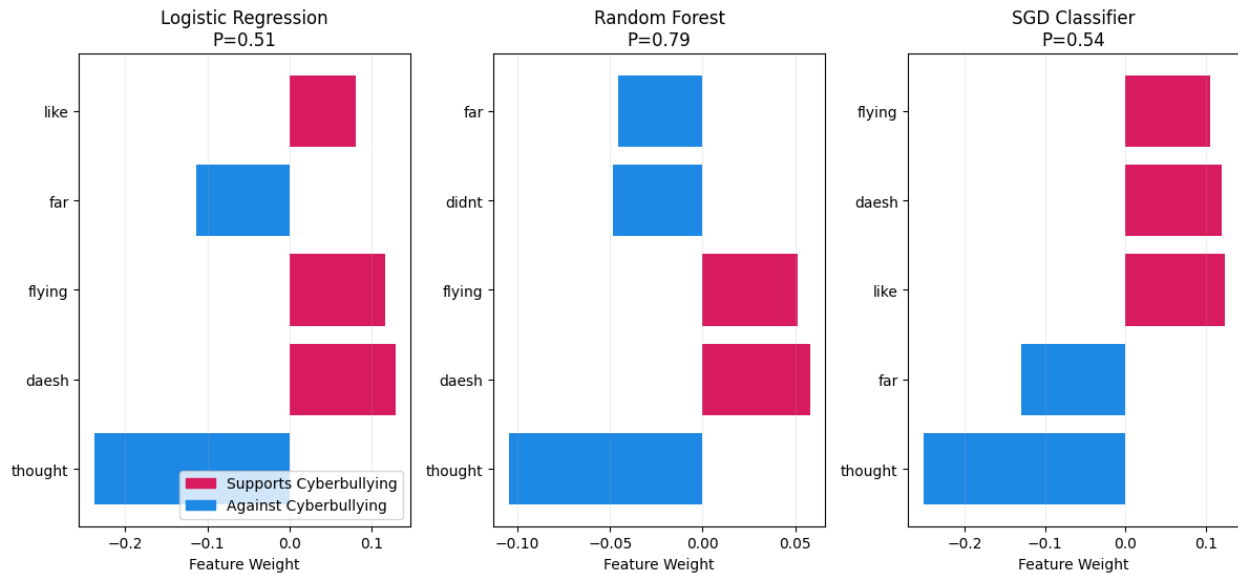


Figure 4.4: Machine Learning LIME Explanations Case 1

Observing figure 4.4 we see that all models demonstrated confidence in their predictions in this instance involving phrases like "daesh" (referring to ISIS) (P=0.51 to 0.79). The most reliable and consistent feature interpretation was shown by Random Forest (P=0.79). Words like "daesh" and "flying" were found to promote the classification of cyberbullying across all models, whereas words like "thought" consistently counted against the classification of cyberbullying. The word "far" was interpreted differently by each model; Random Forest viewed it as supportive of cyberbullying, whereas Logistic Regression and SGD classified it as anti-bullying. This inconsistency emphasises how difficult it is for machine learning models to interpret unclear data.

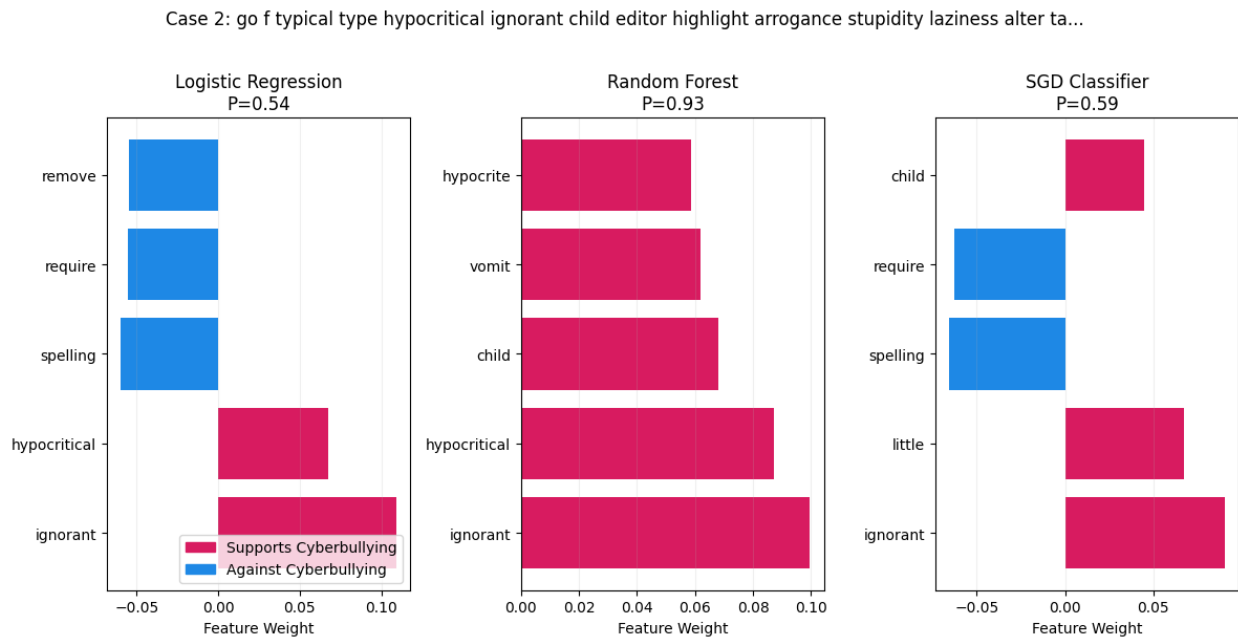


Figure 4.5: Machine Learning LIME Explanations Case 2

In figure 4.5, there were more phrases like "hypocritical," "ignorant," and "child." In contrast to SGD ($P=0.59$) and LR ($P=0.54$), RF ($P=0.93$) demonstrated exceptionally high confidence. The LIME explanations of the models show notable variations in the elements they prioritised. All potentially offensive terms that enable cyberbullying were successfully detected by Random Forest, however Logistic Regression and SGD had more ambiguous interpretations, missing certain disparaging terms. All models properly detected this explicit insult, as seen by the continuous identification of the term "ignorant" as the strongest indication of cyberbullying.

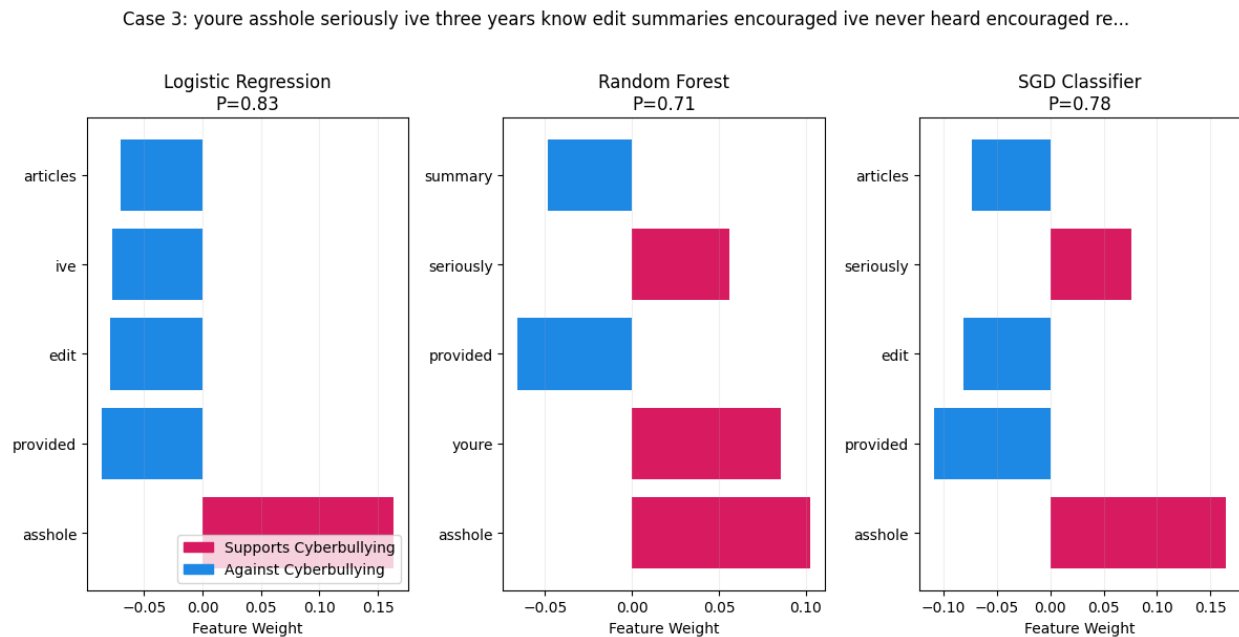


Figure 4.6: Machine Learning LIME Explanations Case 3

As we can see in figure 4.6, there was a direct personal insult in this instance, using very explicit words. The most confidence in categorising this as cyberbullying was demonstrated by Logistic Regression ($P=0.83$), while all models demonstrated a comparatively high level of confidence ($P=0.71$ to 0.83). Across all models, the explicit slurs were unanimously found to be the best indicator of cyberbullying, proving that they are capable of identifying overt harassment. The models' interpretations of context words varied, though, with Logistic Regression giving neutral keywords like "edit" and "provided" greater weight as evidence against the classification of cyberbullying, demonstrating that it comprehended the conversation context more thoroughly than just the slur.

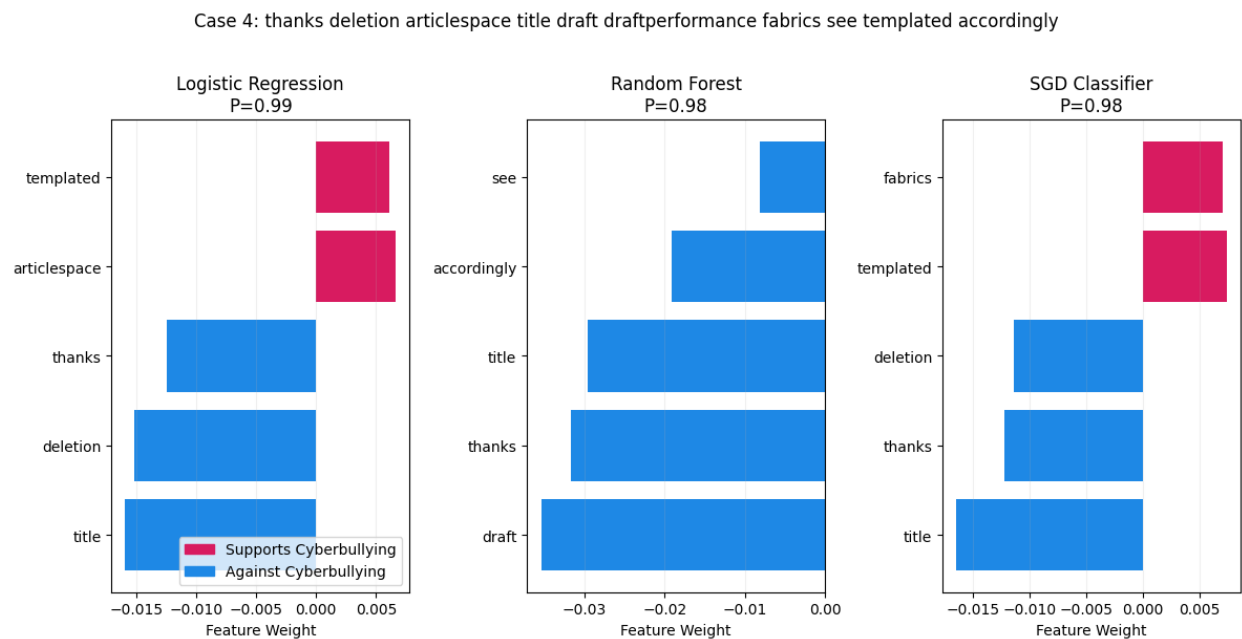


Figure 4.7: Machine Learning LIME Explanations Case 4

The majority of the text in figure 4.7 was neutral and discussed article editing. All models accurately classified this as non-cyberbullying, demonstrating exceptionally high confidence (P=0.98 to 0.99) in their classifications. Words like "thanks," "deletion," and "title" were consistently assessed as evidence against cyberbullying across models, according to the LIME explanations. This illustrates how well the models can decipher neutral administrative language. It's interesting to note that Logistic Regression and SGD identified phrases like "templated" and "articlespace" as being supportive of cyberbullying. This could suggest that the models overfitted to specific technical terms that were used in cyberbullying situations in the training data.

4.4.2. SHAP Analysis in Machine Learning Models

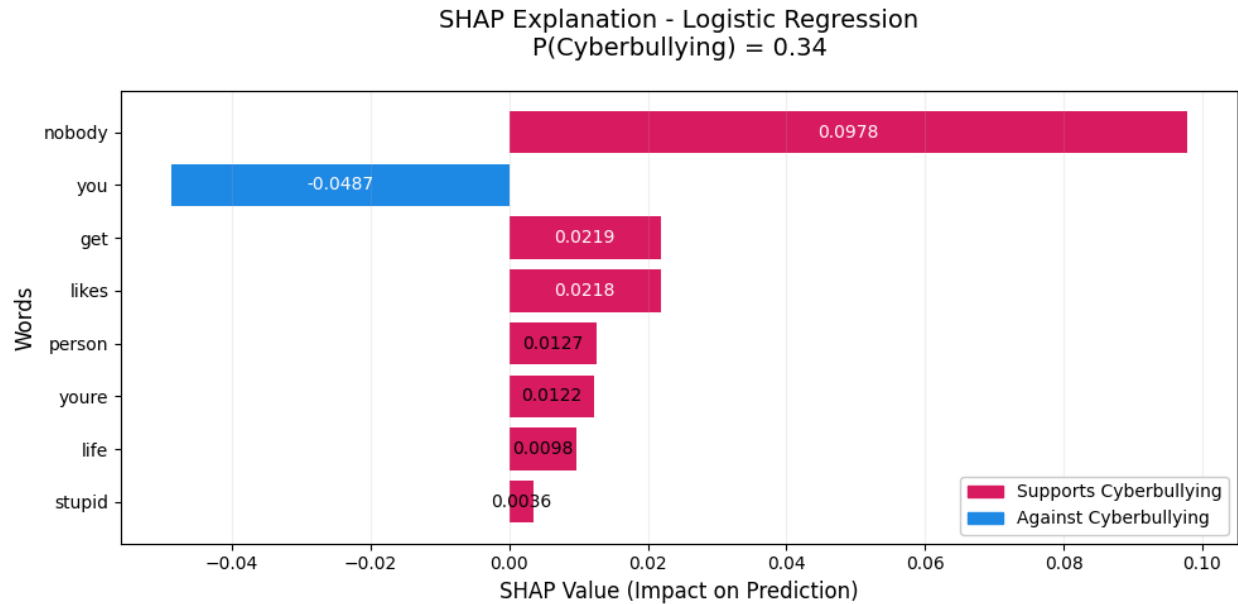


Figure 4.8: SHAP Explanations for Logistic Regression

A moderate level of confidence that the text contains cyberbullying content is indicated by the Logistic Regression model's 0.34 cyberbullying probability in figure 4.8. The term "nobody," which has a SHAP value of 0.0978, is the most influential attribute that supports cyberbullying. This implies that the model strongly links this term to statements that are contemptuous or disparaging. With a negative SHAP score of -0.0487, the word "you" interestingly works against the cyberbullying classification, suggesting that the model might read direct address differently than third-person references. Words like "get," "likes," "person," "youre," "life," and "stupid" also had favourable effects on the cyberbullying classification, but their impact values are substantially lower, ranging from 0.0086 to 0.0219.

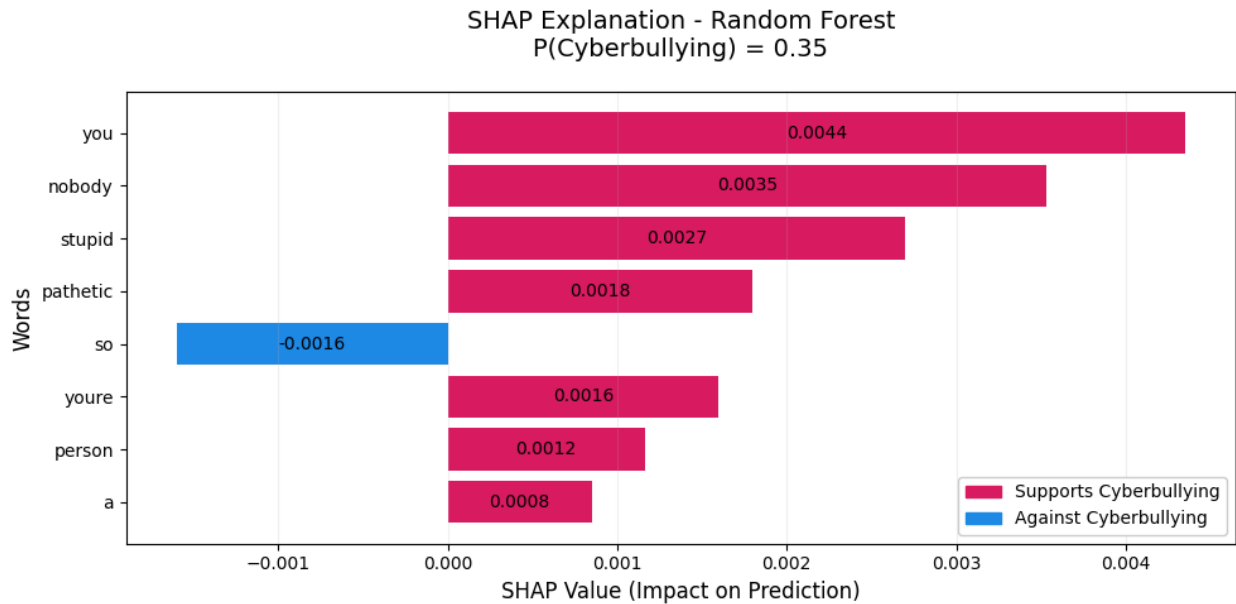


Figure 4.9: SHAP Explanations for Random Forest

With a cyberbullying probability of 0.35, the Random Forest classifier displays a similar evaluation in figure 4.9; however, it exhibits a distinct feature importance pattern. With a SHAP value of 0.0044, Random Forest finds "you" to be the greatest indication supporting cyberbullying, in contrast to Logistic Regression, indicating a discrepancy in how these models understand pronouns. With a value of 0.0035, "Nobody" is ranked second in significance but supports cyberbullying. Cyberbullying is influenced by phrases like "stupid" (0.0027), "pathetic" (0.0018), and "youre" (0.0016), which are all equally important according to the Random Forest model. With a value of -0.0016, only the phrase "so" is detrimental to the classification. The fact that the SHAP values are much less than those of the other models suggests that more intricate patterns are captured by Random Forest's ensemble nature.

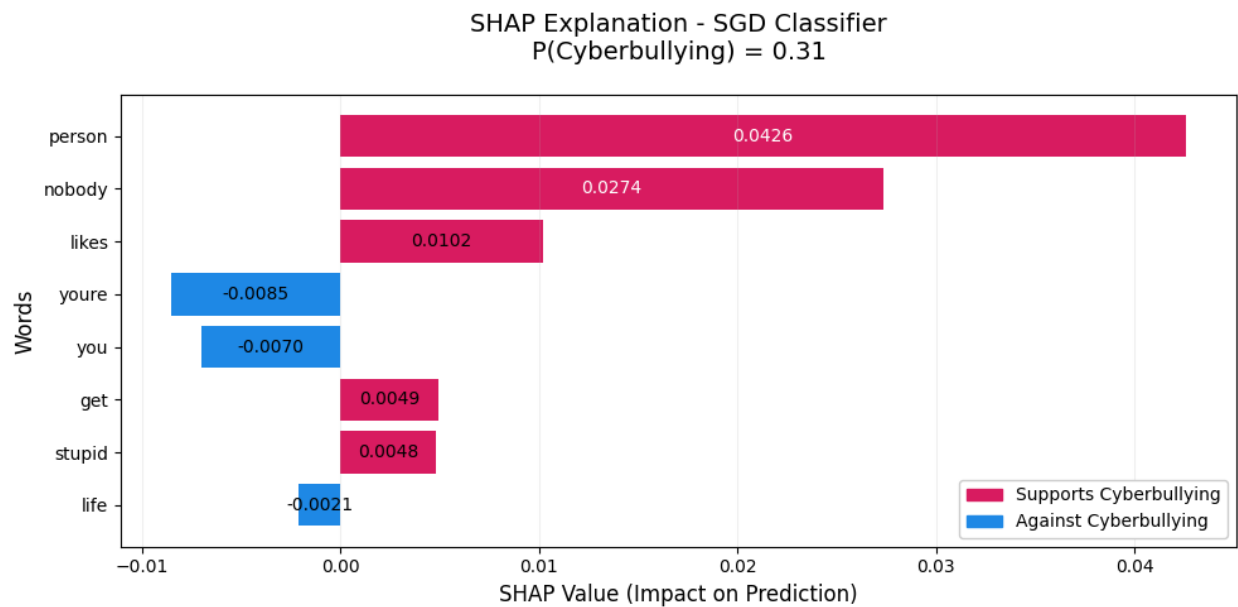


Figure 4.10: SHAP Explanations for SGD Classifier

With a likelihood of 0.31 for cyberbullying, the SGD Classifier shows yet another clear feature attribution trend in figure 4.10. With a high SHAP value of 0.0426, this model identifies "person" as the most reliable indication of cyberbullying, followed by "nobody" at 0.0274. With negative SHAP values of -0.0070 and -0.0085, respectively, SGD sees "you" and "youre" as evidence against cyberbullying, in contrast to Random Forest but comparable to Logistic Regression. Additionally, the term "life" works against the classification of cyberbullying (-0.0021). Even when trained on the same data and producing identical overall predictions, different optimisation techniques can result in essentially distinct feature interpretations, as seen by the inconsistent personal pronoun interpretation between SGD and the other models.

4.4.3. LIME Analysis in Deep Learning Models

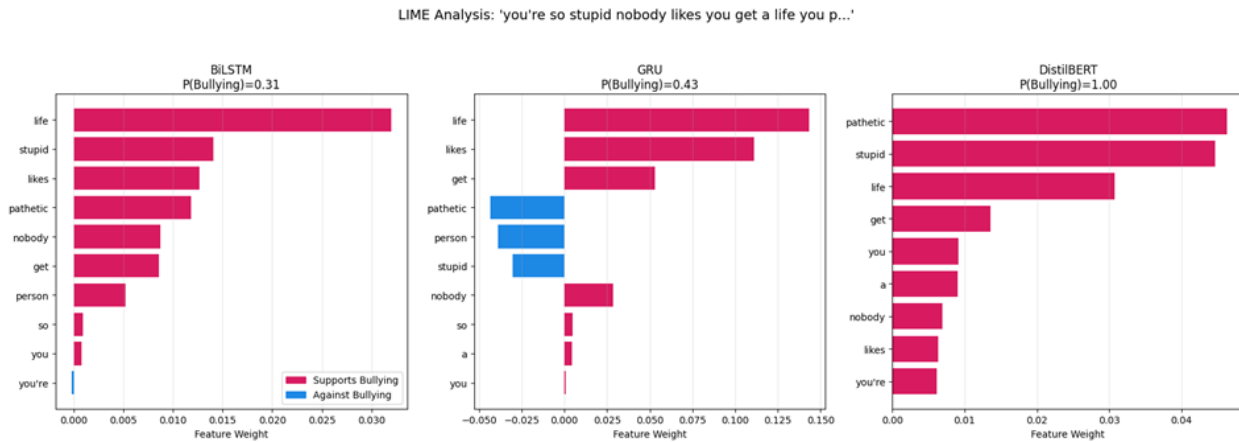


Figure 4.11: LIME Explanations for Deep Learning Models

There are notable variations amongst the models in terms of feature interpretation and confidence levels in figure 4.11. With probabilities of 0.31 and 0.43, respectively, BiLSTM and GRU exhibit moderate confidence, whereas DistilBERT exhibits exceptional confidence with a bullying probability of 1.00. Interesting model-specific interpretations are revealed by the feature weightings. "Life" is given the most weight in BiLSTM when it comes to bullying, followed by "stupid" and "likes." Although "life" is given the highest weight in the GRU model, it is interpreted more fairly, with phrases like "pathetic," "person," and "stupid" weighing against the bullying classification. With its transformer architecture, DistilBERT finds that practically every phrase encourages bullying, with "pathetic" and "stupid" having the biggest impact. The fact that transformer-based models seem more sensitive to the overall negative environment than recurrent neural networks demonstrates how architectural changes significantly affect interpretation.

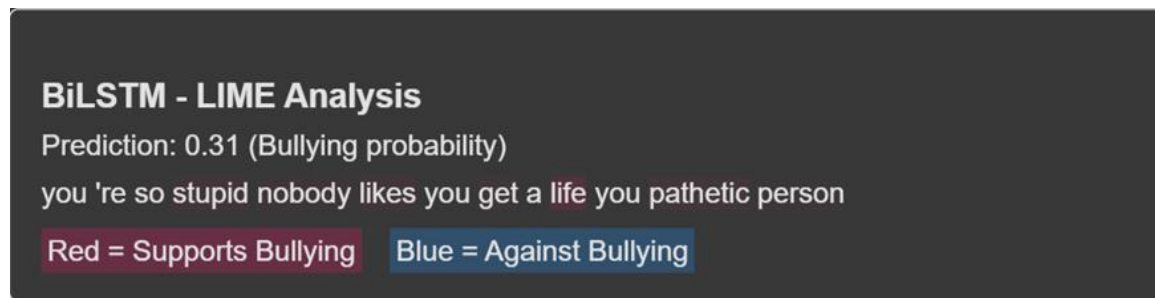


Figure 4.12: Bi-LSTM - LIME Analysis

The cyberbullying text observed in figure 4.12 has a moderate bullying probability of 0.31, according to the BiLSTM model analysis. With comparatively balanced weights, this recurrent neural network design finds important offensive phrases as supporting evidence for bullying classification. Words like "life," "stupid," "likes," and "pathetic" are the most influential, and the model emphasises them in various hues of red to show support for bullying classification. It's interesting to note that personal pronouns like "you" and "you're" are given very little weight, indicating that the BiLSTM is more concerned with the negative tone of the language than with the directed nature of the attack. The model isn't totally certain that this is cyberbullying, as indicated by the moderate overall probability. This could be because of its sequential processing technique, which might not adequately capture the attack's whole context.

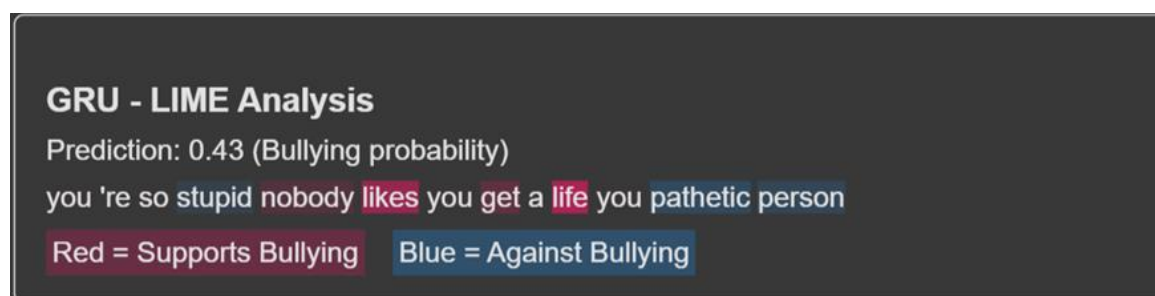


Figure 4.13: GRU - LIME Analysis

Compared to BiLSTM, the GRU model gives the sentence a marginally higher bullying probability of 0.43, indicating a moderate level of confidence seen in figure 4.13. Surprisingly, "pathetic," "person," and "stupid" work against bullying classification (highlighted in blue), whereas "life" and "likes" substantially promote bullying

classification (highlighted in red). This illustrates a complex pattern of interpretation. This unexpected finding raises the possibility that the GRU is utilising the imperative phrase "get a life" as a bullying indicator even though they may have learnt the other bad phrases in non-bullying settings. Although the results of this balanced approach may be easier to understand, they seem to be less in line with human intuition on the terms that make up bullying language.

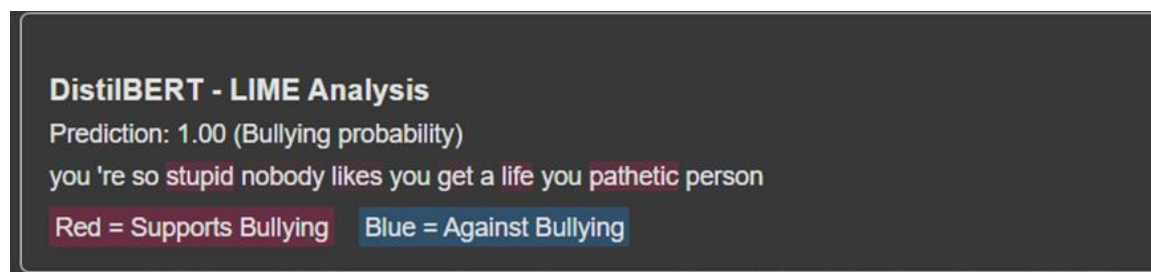


Figure 4.14: DistilBERT - LIME Analysis

With a bullying probability of 1.00, the DistilBERT transformer model exhibits 100% certainty, greatly surpassing the confidence of both recurrent models as seen in figure 4.14. DistilBERT identifies "pathetic," "stupid," and "nobody likes you" as the most potent markers of bullying material in the visualisation. It acknowledges the cumulative effect of several negative words and seems to better represent the message's overall context than the recurrent models. Transformer-based models may be better at spotting complex kinds of cyberbullying that depend on contextual understanding rather than merely identifying single negative phrases, according to this thorough interpretation and high confidence level.

4.4.4. SHAP Analysis in Deep Learning Models

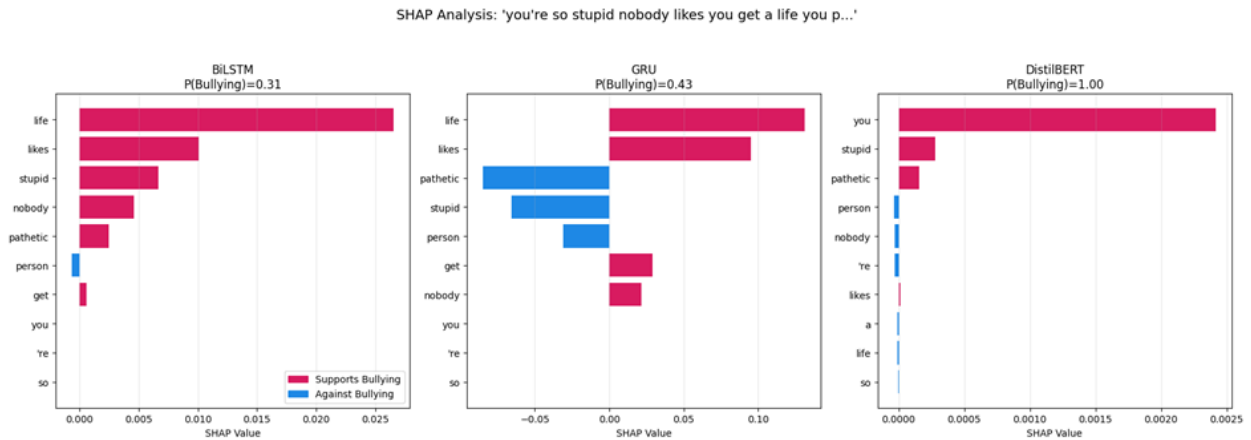


Figure 4.15: SHAP Explanations for Deep Learning Models

Figure 4.15 offers quantitative insights into feature relevance by comparing the SHAP values across all three models. The most powerful positive influence on bullying classification for BiLSTM is "life" (~ 0.025), followed by "likes" (~ 0.015) and "stupid" (~ 0.01). The surprising pattern observed in the preceding visualisation is confirmed by the GRU model, which also significantly weights "life" (~ 0.1) yet displays strong negative weights for "pathetic" and "stupid" (both about -0.05). "You" is the most impactful term (~ 0.02) according to DistilBERT, while "stupid" has a significantly lower positive influence (~ 0.0005). The models' scale differences are instructive: DistilBERT has very small values (± 0.002), whereas GRU has the biggest magnitude values (± 0.1). This suggests that DistilBERT's confidence is derived from the cumulative effect of several little contributions rather than a limited number of strongly weighted elements.

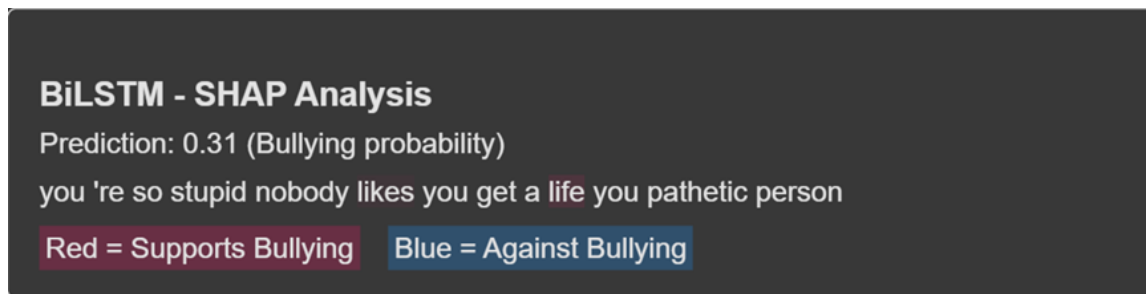


Figure 4.16: Bi-LSTM - SHAP Analysis

The example text in figure 4.16 receives a bullying probability of 0.31 from the BiLSTM model, indicating a rather low level of categorisation confidence. Although the precise contribution levels are not measured in this graphic, the SHAP visualisation uses colour coding to highlight a number of important phrases. In contrast to other terms that operate against the bullying classification (highlighted in blue), words like "stupid," "nobody," and "likes" seem to favour it (highlighted in red). Interestingly, the comparatively low confidence score may be the result of the BiLSTM's sequential processing, which may hinder its capacity to integrate the cumulative impact of numerous negative words together, even when the entire text is obviously offensive. Although it doesn't measure each word's specific contribution, the visualisation offers a qualitative understanding of which terms the model is concentrating on.

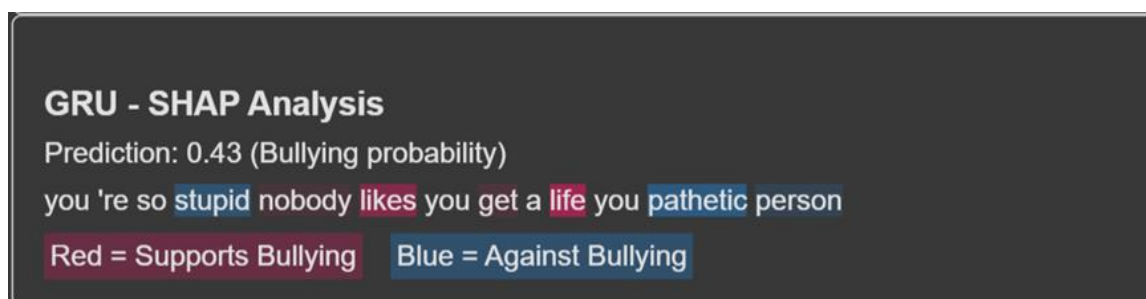


Figure 4.17: GRU - SHAP Analysis

With a bullying probability of 0.43, the GRU model has a somewhat higher level of confidence. A more intricate pattern of interpretation is visible in this SHAP visualisation. While "stupid," "pathetic," and "person" are highlighted in blue, implying they help against bullying categorisation, "likes" and "life" are highly highlighted in red, suggesting

they encourage bullying classification. The GRU model may have picked up odd connections during training by coming across these negative phrases in non-bullying contexts, as evidenced by this contradictory attribution pattern. It is still not reading the whole context in a way that is consistent with human judgement regarding inappropriate language, as evidenced by the mixed feature attributions.

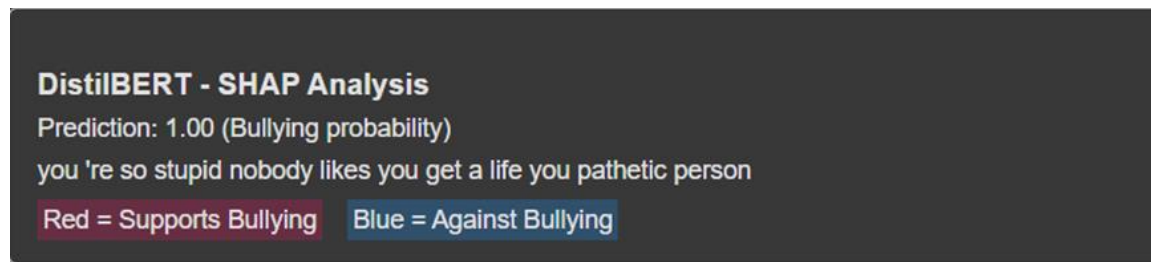


Figure 4.18: DistilBERT - SHAP Analysis

With a bullying probability of 1.00, the DistilBERT model exhibits complete certainty, showcasing the transformer's exceptional contextual awareness as seen in figure 4.18. We can observe that DistilBERT seems to be underlining portions of the text in red, suggesting support for the bullying classification, even if this visualisation does not quantitatively show the extent of word-level highlighting. The self-attention mechanisms of the transformer architecture probably allow it to better comprehend word associations and identify the cumulative effect of several negative sentences. Because DistilBERT is better at integrating the entire context of the message rather than processing it sequentially like the recurrent models, it may be able to recognise this as a clear-cut case of cyberbullying, as indicated by the perfect confidence score.

4.4.5. Attention Visualization Analysis for DistilBERT Model

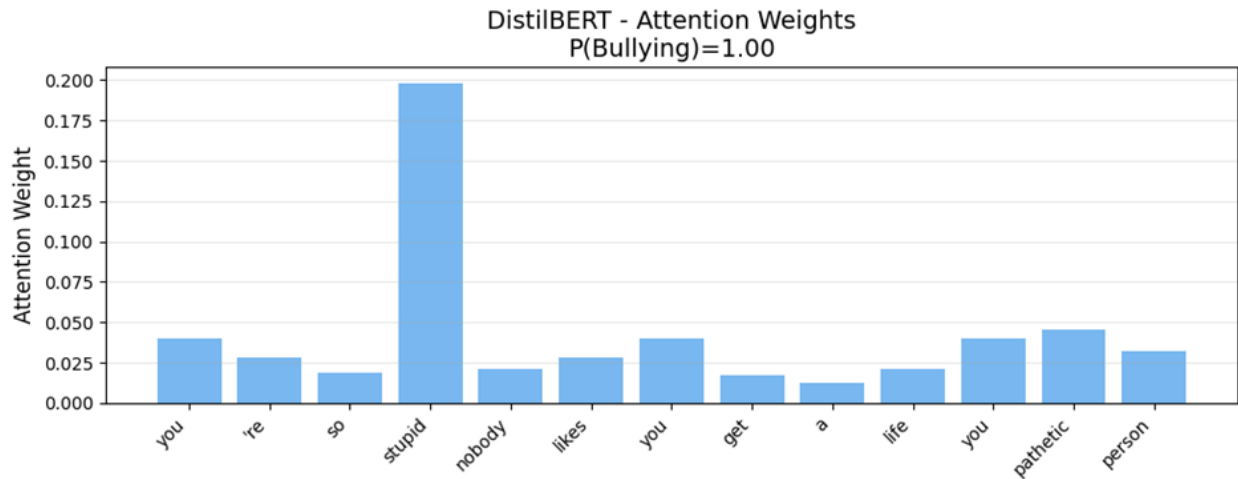


Figure 4.19: DistilBERT - Attention Visualization

In figure 4.19, a remarkable pattern in the transformer model's processing of cyberbullying text is revealed by the DistilBERT attention weights visualisation, which assigns a perfect probability score of 1.00. Most significantly, the word "stupid" gets about four times as much attention as the next highest term in the text, receiving almost 0.200 times as much attention as any other word. This overpowering emphasis implies that DistilBERT has mastered the ability to recognise this overt insult as a key sign of cyberbullying content. The remaining words receive secondary attention more evenly, with "pathetic" and "you" tokens receiving somewhat greater weights (around 0.045 and 0.040, respectively). DistilBERT's perfect confidence score can be explained by the attention distribution; the model reinforces its classification by heavily weighting the most blatantly insulting phrase while also taking into account the context of aimed insults.



Figure 4.20: DistilBERT - Attention Analysis

By displaying attention directly on the text as observed in figure 4.20, this visualisation enhances the preceding bar chart. "Stupid" is highlighted in dark blue to denote high attention. It is evident that the explicit insult "stupid" is the main focus of the model's attention mechanism, which highlights other textual elements with lighter blue, signifying less attention. This graphic demonstrates that even though the statement contains several objectionable words ("pathetic," "nobody likes you," "get a life"), DistilBERT's focus is mostly focused on the term that is most overtly disparaging. The model has enough confidence to classify with certainty thanks to the perfect bullying likelihood score (1.00), which indicates focused attention on a single offensive term and distributed attention over the entire message.

4.5. Results & Implementation Summary

In the goal of cyberbullying detection, the experimental evaluation unequivocally demonstrates the advantages and disadvantages of both sophisticated deep learning architectures and conventional machine learning models. When combined with Word2Vec embeddings, machine learning models including Random Forest, Logistic Regression, and SGD demonstrated good baseline performance; Random Forest had the highest F1-score of all of them. Deep learning models, especially Bi-LSTM, outperformed them in the end, with the maximum accuracy of 92.16% and continuously high F1-scores, proving that it could recognise intricate sequential patterns in text.

In this experimental environment, DistilBERT scored somewhat lower F1-scores than Bi-LSTM, despite being theoretically predicted to perform better than recurrent networks because of its transformer-based architecture. Differences in data sensitivity or less-than-

ideal hyperparameter selection may be to blame for this. However, the attention visualisation analysis revealed targeted prioritisation of important bullying cues, indicating that DistilBERT had superior contextual knowledge.

Applying Explainable AI (XAI) approaches, such as LIME, SHAP, and attention visualisations, gave model decision-making crucial transparency. While DistilBERT demonstrated excellent contextual attention mechanisms that significantly matched human intuition for cyberbullying detection, Random Forest consistently delivered the clearest feature attributions in LIME and SHAP evaluations across machine learning models.

In conclusion, deep learning architectures performed better in detecting cyberbullying, especially Bi-LSTM and DistilBERT. Traditional models, on the other hand, required less computing power and provided better interpretability. The black-box aspect of complicated models was effectively addressed by the incorporation of XAI approaches, improving model transparency and fostering confidence in AI-assisted moderation systems. These results highlight the need for future cyberbullying detection frameworks to aim for a compromise between interpretability and strong prediction performance in order to guarantee accountability, fairness, and user trust.

5. DISCUSSIONS

5.1. Overview

The outcomes of applying machine learning, deep learning, and explainable AI algorithms for cyberbullying detection are covered in this chapter. The results are critically examined in light of previous research, emphasising similarities, advancements, and differences. It also talks about the limitations of the existing work and its implications for morality.

5.2. Machine Learning Models Discussion

SGD, Random Forest, and Logistic Regression were the machine learning models that produced competitive baseline results; Random Forest had the highest F1-score, at 81%. This result supports earlier research, like (Sultan et al., 2023), which emphasised the effectiveness of ensemble techniques like Random Forest for text-based cyberbullying identification because of their resilience to noise and unbalanced data.

Nonetheless, our models displayed weaknesses that were comparable to those noted by (Dadvar et al., 2013), specifically sensitivity to changing slang and sarcasm. Despite being interpretable, logistic regression has trouble adjusting to indirect or coded language, which was another issue brought up by (Chatzakou et al., 2017) when employing models based on keywords and metadata.

Our results show notable improvements over previous studies like (Reynolds et al., 2011), which achieved about 70–75% accuracy in early machine learning-based detection efforts. This is mainly because of improved preprocessing (e.g., word embeddings) and more advanced feature engineering strategies. However, the problem of contextual ambiguity—in which harmless remarks were mistakenly reported—remains, confirming the restrictions mentioned by (Zhao et al., 2016).

5.3. Deep Learning Models Discussion

The results of (Hasan et al., 2023), who highlighted that deep architectures provide improved context awareness by automatically learning hierarchical representations, were corroborated by the considerable outperformance of deep learning models over machine learning models.

Bi-LSTM outperformed transformer-based DistilBERT in our testing, achieving the maximum F1-score of 86% and an accuracy of 92.16% each time. This is somewhat in contrast to (Chen et al., 2024), who used bidirectional transformers to improve the performance of a hybrid XLNet-BiLSTM model over pure LSTM. Our findings imply that the memory structure of Bi-LSTM is still quite effective for cyberbullying datasets that place a significant emphasis on the sequential accumulation of toxicity.

Even though DistilBERT performed well (F1-score of 84%), it was unable to outperform Bi-LSTM. Like (Ejaz et al., 2024), who highlighted that transformers require careful domain adaptation to maximise benefits, our DistilBERT model may need significant fine-tuning to adequately catch bullying cues that are platform-specific or driven by sarcasm.

Intriguingly, (Agrawal and Awekar, 2018) also observed that although deep learning models manage direct harassment better, lengthy, context-dependent interactions continue to provide difficulties—consistent with our observations.

Therefore, our findings support the trend observed by several research that, even while transformers are strong, specialised designs like Bi-LSTM may still be very helpful for domain-specific tasks like cyberbullying detection when temporal patterns are important.

5.4. Explainable AI (XAI) Techniques Discussion

In order to analyse model behaviour beyond accuracy metrics, explainable AI was essential. In line with the trends in responsible AI covered by (Lundberg and Lee, 2017) and (Ribeiro et al., 2016), techniques including SHAP, LIME, and attention visualisation were used.

In terms of providing quick and understandable explanations, LIME worked especially well for conventional models, which is consistent with findings from (Ribeiro et al., 2016). According to our findings, LIME successfully identified localised features in Random Forest predictions, which helped to clarify misclassifications.

SHAP provided reliable local and worldwide interpretability. (Alvarez-Melis and Jaakkola, 2018) state that SHAP assisted in revealing hidden biases, such as the model's excessive use of strong negative phrases without context—a feature that is essential for the ethical deployment of AI.

Consistent with (Vaswani et al., 2017), Attention Visualisation for DistilBERT demonstrated that the model prioritised negative transitional phrases ("not even joking," "you suck") in all sentence circumstances. Though our results also concur with (Blodgett et al., 2020), who warn against assuming attention weights fully explain model reasoning, this discovery shows a good trend towards making deep models more explainable.

Overall, XAI techniques not only increased model transparency but also revealed flaws, like sensitivity to common but non-aggressive words—a problem that (Binns et al., 2018) also brought out in relation to explainability issues.

5.5. Ethical & Practical Implications

Our research confirms what (Espelage and Hong, 2016) have emphasised: high accuracy by itself is not enough for real-world cyberbullying detection systems. Deployment that is ethical needs:

Transparency: Moderator trust was increased and concerns about algorithmic bias were diminished as a result of LIME and SHAP explanations.

Identification of Biases: SHAP identified possible biases, bolstering (Selbst and Barocas's, 2018) claim that unexplained black-box AI systems may inadvertently reinforce discrimination.

Adaptability to Linguistic Drift: Our results confirm that models require ongoing retraining to accommodate new slang, memes, and sarcasm, which change quickly on social media platforms. This is in line with findings by (Ejaz et al., 2024) and (Zampieri et al., 2019).

Therefore, for ethical and trustworthy cyberbullying detection, XAI and deep learning must be combined.

5.6. Limitations of Study

Although the results are encouraging, it is important to recognise the limitations:

Dataset Limitations: Similar to (Xu et al., 2012), our dataset was primarily in English, which limited its generalisability to multilingual or code-switched discussions.

Computational Restrictions: In accordance with (Strubell et al., 2019), it is crucial to highlight that fine-tuning transformers have high resource requirements, which prevent widespread deployment.

Evaluation of Interpretability: Although explanation tools were used, results should be strengthened by formal user tests that gauge human trust and comprehension, as suggested by (Binns et al., 2018).

These constraints should be addressed in future research to promote scalable and responsible cyberbullying detection.

5.7. Discussions Summary

To sum up, the discussion confirms that Bi-LSTM and DistilBERT models provide a strong and comprehensible strategy for detecting cyberbullying when paired with XAI techniques like SHAP and LIME. Our findings support current literature trends, but they also point to shortcomings in generalisability, ethical considerations, and handling

changing language, highlighting the necessity of continued research towards AI systems that are transparent, equitable, and flexible.

6. CONCLUSION & FUTURE WORK

6.1. Conclusion

In order to improve interpretability and ethical trust, this study combined Explainable AI (XAI) techniques with both classical and deep learning models to advance cyberbullying detection. The study showed through systematic evaluation that deep learning models, especially Bi-LSTM and DistilBERT, offer notable improvements in accuracy, F1-score, and the capacity to capture subtle, context-dependent bullying, while machine learning models, such as Logistic Regression and Random Forest, achieve respectable performance.

With an F1-score of 86%, the Bi-LSTM model in particular performed the best, underscoring the significance of sequential dependency modelling in identifying progressive harassing patterns. DistilBERT confirmed the promise of transformer designs in cyberbullying detection, in line with current research trends, albeit being marginally less accurate in this investigation (Chen et al., 2024; Ejaz et al., 2024).

A significant weakness in cyberbullying detection systems was filled by the incorporation of XAI techniques—LIME, SHAP, and attention visualization—which made model conclusions understandable and transparent. These methods allowed for both technical and ethical review by exposing significant insights into model behaviour, biases, and misclassification tendencies. Calls for responsible AI development in delicate applications, such as online content moderation, are in line with this transparency (Binns et al., 2018; Lundberg & Lee, 2017).

Overall, this study demonstrates that in order to guarantee trust, equity, and accountability in AI-based cyberbullying detection systems, strong explainability mechanisms must be used in conjunction with good classification performance.

6.2. Limitations

Significant outcomes were obtained, although there were a number of drawbacks:

Dataset Limitations: The dataset's generalisability was limited by its preponderance of English-based data and lack of multilingual or cross-cultural diversity.

Computational Resources: DistilBERT's performance cap may have been impacted by hardware constraints that prevented extensive fine-tuning of huge transformer models.

Evaluation of Interpretability: More structured user tests measuring end-user happiness and trust were not carried out; instead, interpretability evaluation was mostly based on qualitative analysis.

Handling Temporal Drift: Because the models were static, they were not assessed for dynamic adaptation to new slang or changes in language over time.

6.3. Future Work

The following directions are suggested for further study:

Multilingual and Cross-Platform Detection: Global applicability will be improved by expanding model capabilities to address multilingual cyberbullying, including code-switching and culturally distinctive expressions.

Continual Learning Techniques: Models can be made to adjust to changing slang and online language by putting dynamic retraining or continual learning frameworks into practice.

Expanded Explainability Evaluation: To evaluate the utility, credibility, and cognitive load of explanations, user studies with moderators and regular users should be carried out.

Bias Mitigation Strategies: To lessen the disproportionate impact on particular communities, future research should concentrate on debiasing models during training utilising fairness-aware learning techniques.

Deploying good models in low-resource environments: This can be facilitated by investigating lightweight architectures or knowledge distillation techniques.

Human-in-the-Loop Moderation Systems: A balance between automated detection efficiency and sophisticated judgement may be achieved by combining automated detection with human review interfaces, bolstered by XAI explanations.

7. REFERENCES

- Atske, S. (2022) *Teens and Cyberbullying 2022*. Pew Research Center. Available at: <https://www.pewresearch.org/internet/2022/12/15/teens-and-cyberbullying-2022/> (Accessed: 28 April 2025).
- Agrawal, S. and Awekar, A. (2018) 'Deep Learning for Detecting Cyberbullying Across Multiple Social Media Platforms', *Lecture Notes in Computer Science*, pp. 141–153. doi: https://doi.org/10.1007/978-3-319-76941-7_11
- Alvarez-Melis, D. and Jaakkola, T.S. (2018) *Towards Robust Interpretability with Self-Explaining Neural Networks*. arXiv. Available at: <https://arxiv.org/abs/1806.07538> (Accessed: 23 April 2025).
- Blodgett, S.L., Barocas, S., Daumé III, H. and Wallach, H. (2020) *Language (Technology) is Power: A Critical Survey of 'Bias' in NLP*. arXiv. Available at: <https://arxiv.org/abs/2005.14050> (Accessed: 23 April 2025).
- Binns, R. et al. (2018) 'It's Reducing a Human Being to a Percentage', *arXiv (Cornell University)*, pp. 1–14. doi: <https://doi.org/10.1145/3173574.3173951>
- Centers for Disease Control and Prevention (CDC) (2025) *Facts About Suicide*. Available at: <https://www.cdc.gov/suicide/facts/index.html> (Accessed: 28 April 2025).
- Chen, S., Wang, J. and He, K. (2024) 'Chinese Cyberbullying Detection Using XLNet and Deep Bi-LSTM Hybrid Model', *Information*, 15(2), p. 93. doi: <https://doi.org/10.3390/info15020093>
- Chatzakou, D., Kourtellis, N., Blackburn, J., De Cristofaro, E., Stringhini, G. and Vakali, A. (2017) 'Measuring #GamerGate', *Proceedings of the 26th International Conference on World Wide Web Companion - WWW '17 Companion*, pp. 1285–1290. doi: <https://doi.org/10.1145/3041021.3053890>
- Dennehy, R., Meaney, S., Cronin, M. and Arensman, E. (2020) 'The psychosocial impacts of cybervictimisation and barriers to seeking social support: Young people's perspectives', *Children and Youth Services Review*, 111, p. 104872. doi: <https://doi.org/10.1016/j.childyouth.2020.104872>

Doshi-Velez, F. and Kim, B. (2017) *Towards a Rigorous Science of Interpretable Machine Learning*. arXiv. Available at: <https://arxiv.org/abs/1702.08608> (Accessed: 28 April 2025).

Dadvar, M., Trieschnigg, D., Ordelman, R. and de Jong, F. (2013) 'Improving Cyberbullying Detection with User Context', *Lecture Notes in Computer Science*, pp. 693–696. doi: https://doi.org/10.1007/978-3-642-36973-5_62

Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2018) *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv. Available at: <https://arxiv.org/abs/1810.04805> (Accessed: 23 April 2025).

Ejaz, N., Razi, F. and Choudhury, S. (2024) 'Towards comprehensive cyberbullying detection: A dataset incorporating aggressive texts, repetition, peerness, and intent to harm', *Computers in Human Behavior*, 153, p. 108123. doi: <https://doi.org/10.1016/j.chb.2023.108123>

Espelage, D.L. and Hong, J.S. (2016) 'Cyberbullying Prevention and Intervention Efforts: Current Knowledge and Future Directions', *The Canadian Journal of Psychiatry*, 62(6), pp. 374–380. doi: <https://doi.org/10.1177/0706743716684793>

Hasan, M.T., Emran, A., Hossain, S., Akter, A., Ahmed, M. and Islam, S. (2023) 'A Review on Deep-Learning-Based Cyberbullying Detection', *Future Internet*, 15(5), p. 179. doi: <https://doi.org/10.3390/fi15050179>

Karthik Dinakar, Reichart, R. and Lieberman, H. (2021). Modeling the Detection of Textual Cyberbullying. *Proceedings of the International AAAI Conference on Web and Social Media*, 5(3), pp.11–17. doi:<https://doi.org/10.32604/cmc.2023.033682>

Li, X., Liu, T. and Liu, Y. (2019) 'Cause Analysis of Unsafe Behaviors in Hazardous Chemical Accidents: Combined with HFACs and Bayesian Network', *International Journal of Environmental Research and Public Health*, 17(1), p. 11. doi: <https://doi.org/10.3390/ijerph17010011>

Liu, Y. et al. (2019) *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv. Available at: <https://arxiv.org/abs/1907.11692> (Accessed: 23 April 2025).

- Lundberg, S. and Lee, S.-I. (2017) 'A Unified Approach to Interpreting Model Predictions'. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf
- Lundberg, S.M., Erion, G.G. and Lee, S.-I. (2018) *Consistent Individualized Feature Attribution for Tree Ensembles*. arXiv. Available at: <https://arxiv.org/abs/1802.03888> (Accessed: 23 April 2025).
- Nixon, C. (2014) 'Current perspectives: the impact of cyberbullying on adolescent health', *Adolescent Health, Medicine and Therapeutics*, p. 143. doi: <https://doi.org/10.2147/ahmt.s36456>
- Reynolds, K., Kontostathis, A. and Edwards, L. (2011) 'Using Machine Learning to Detect Cyberbullying', pp. 241–244. doi: <https://doi.org/10.1109/icmla.2011.152>
- Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) 'Why Should I Trust You?', pp. 1135–1144. doi: <https://doi.org/10.1145/2939672.2939778>
- Selbst, A.D. and Barocas, S. (2018) 'The intuitive appeal of explainable machines', *Fordham Law Review*, 87, p. 1085.
- Sultan, D., Omarov, B., Kozhamkulova, Z., Kazbekova, G., Alimzhanova, L., Dautbayeva, A., Zholdassov, Y. and Abdrakhmanov, R. (2023). A Review of Machine Learning Techniques in Cyberbullying Detection. *Computers, Materials & Continua*, 74(3), pp.5625–5640. doi: <https://doi.org/10.32604/cmc.2023.033682>
- Semiu Salawu, He, Y. and Lumsden, J. (2017). Approaches to Automated Detection of Cyberbullying: A Survey. *IEEE Transactions on Affective Computing*, 11(1), pp.3–24. doi: <https://doi.org/10.1109/taffc.2017.2761757>
- Strubell, E., Ganesh, A. and McCallum, A. (2019) *Energy and Policy Considerations for Deep Learning in NLP*. arXiv. Available at: <https://arxiv.org/abs/1906.02243> (Accessed: 23 April 2025).
- Serap Kurbanoglu, Boustany, J., Špiranec, S., Grassian, E., Mizrachi, D. and Roy, L. (2016). *Information Literacy: Moving Toward Sustainability*. Springer. Available at: <https://biblio.ugent.be/publication/7010768> [Accessed 23 Apr. 2025].

- Van Hee, C. et al. (2015) 'Automatic detection and prevention of cyberbullying', *International Conference on Human and Social Analytics (HUSO 2015)*, pp. 13–18. IARIA.
- Vaswani, A. et al. (2017) *Attention Is All You Need*. arXiv. Available at: <https://arxiv.org/abs/1706.03762> (Accessed: 23 April 2025).
- Waseem, Z. and Hovy, D. (2016) 'Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter'. doi: <https://doi.org/10.18653/v1/n16-2013>
- Xu, J.-M., Jun, K.-S., Zhu, X. and Bellmore, A. (2012) 'Learning from Bullying Traces in Social Media'. Available at: <https://aclanthology.org/N12-1084.pdf>
- Zhao, R., Zhou, A. and Mao, K. (2016) 'Automatic detection of cyberbullying on social networks based on bullying features', *Proceedings of the 17th International Conference on Distributed Computing and Networking*, pp. 1–6. doi: <https://doi.org/10.1145/2833312.2849567>
- Zhou, Z.-H. and Liu, X.-Y. (2006) 'Training cost-sensitive neural networks with methods addressing the class imbalance problem', *IEEE Transactions on Knowledge and Data Engineering*, 18(1), pp. 63–77. doi: <https://doi.org/10.1109/tkde.2006.17>
- Zampieri, M. et al. (2019) *Predicting the Type and Target of Offensive Posts in Social Media*. arXiv. Available at: <https://arxiv.org/abs/1902.09666> (Accessed: 23 April 2025).

8. APPENDICES

8.1. Gantt Chart

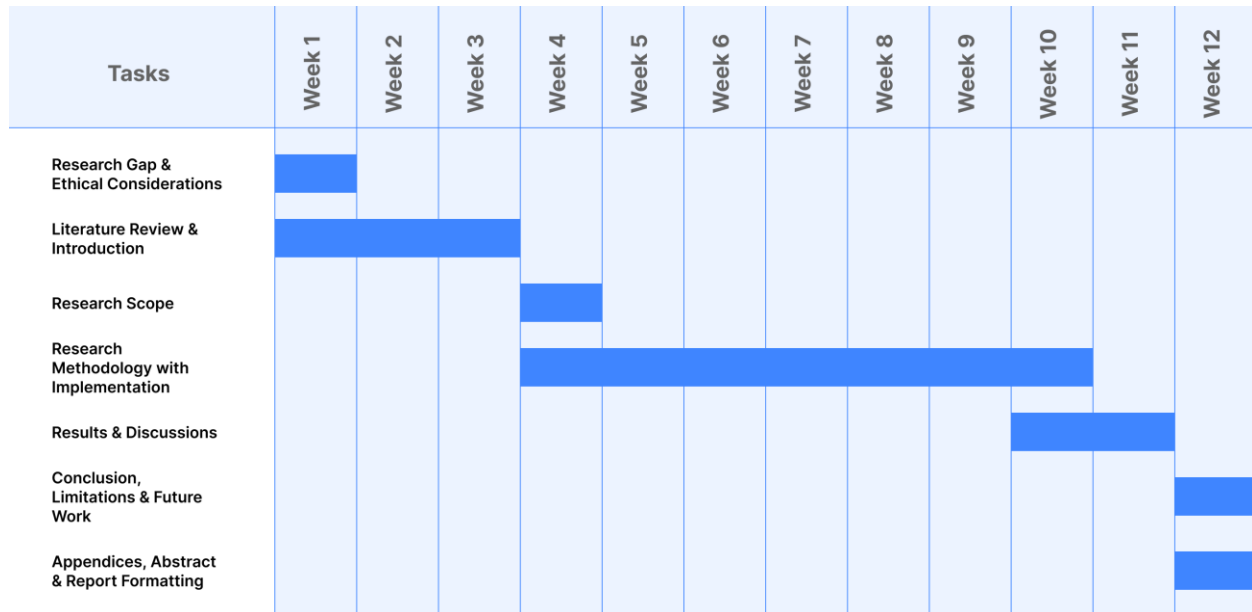


Figure 8.1: Thesis Gantt Chart

Implementation File 1 Code Snippets

```
class LimeExplainer:
```

```
    def __init__(self, models, model_names, word2vec_model, merged_df, y_test, X_train=None,
y_train=None):
```

```
        self.models = models
```

```
        self.model_names = model_names
```

```
        self.word2vec_model = word2vec_model
```

```
        self.merged_df = merged_df
```

```
        self.y_test = y_test
```

```
        self.X_train = X_train
```

```
self.y_train = y_train
self.explainer = lime.lime_text.LimeTextExplainer(
    class_names=["0", "1"], # Match your label encoding
    verbose=True
)

def get_sample_text(self, idx=10):
    """Get a sample text from test data"""
    test_indices = self.merged_df.index[self.merged_df["Label"].isin(self.y_test.tolist())]
    return self.merged_df["Cleaned_Message"].iloc[test_indices[idx]]

def vectorize_text(self, text):
    """Convert text to word2vec embedding"""
    tokens = word_tokenize(text)
    words = [word for word in tokens if word in self.word2vec_model.wv]
    return np.mean(self.word2vec_model.wv[words], axis=0) if words else np.zeros(300)

def predict_proba_wrapper(self, model, texts):
    """Wrapper that ensures consistent label ordering"""
    vectorized_texts = np.array([self.vectorize_text(text) for text in texts])
    probs = model.predict_proba(vectorized_texts.reshape(-1, 300))

    # Ensure probabilities are for classes [0, 1]
    if probs.shape[1] == 2: # Binary classification
        return probs
    else: # Handle potential single-column output
        return np.column_stack([1-probs, probs])

def generate_explanations(self, sample_text, num_features=10):
```

```
"""Generate explanations with label consistency checks"""
explanations = []
for model in self.models:
    try:
        # Calibrate if needed (for SGD)
        if hasattr(model, 'loss') and 'hinge' in str(model.loss) and self.X_train is not None:
            model = CalibratedClassifierCV(model, method='sigmoid').fit(self.X_train,
self.y_train)

        exp = self.explainer.explain_instance(
            sample_text,
            lambda x: self.predict_proba_wrapper(model, x),
            num_features=num_features,
            labels=[0, 1] # Explicitly specify expected labels
        )
        explanations.append(exp)
    except Exception as e:
        print(f"Error explaining {type(model).__name__}: {str(e)}")
        explanations.append(None)
return explanations

def plot_lime_explanation(self, exp, model_name):
    """Robust plotting with updated LIME API"""
    if exp is None:
        print(f"Skipping {model_name} - no explanation generated")
        return

    try:
        # Get explanation for class 1 (Cyberbullying) - no num_features parameter
```

```
words_weights = exp.as_list(label=1)
words, weights = zip(*words_weights)

plt.figure(figsize=(10, 5))
colors = ['#D81B60' if w > 0 else '#1E88E5' for w in weights]
bars = plt.barh(words, weights, color=colors)

# Add prediction info
prob = exp.predict_proba[1]
plt.title(f"{model_name}\nP(Cyberbullying)={prob:.2f}", fontsize=14)
plt.xlabel("Feature Weight", fontsize=12)
plt.ylabel("Words", fontsize=12)
plt.gca().invert_yaxis()

# Add legend
plt.legend(
    [plt.Rectangle((0,0),1,1,color=c) for c in ['#D81B60', '#1E88E5']],
    ['Supports Cyberbullying', 'Against Cyberbullying'],
    loc='lower right'
)
plt.tight_layout()
plt.show()

except Exception as e:
    print(f"Error plotting {model_name}: {str(e)}")

def visualize_all_explanations(self, explanations):
    """Visualize all explanations without num_features"""
    for exp, name in zip(explanations, self.model_names):
```

```
self.plot_lime_explanation(exp, name)

# Generate and view explanations
sample_text = lime_explainer.get_sample_text(idx=10) # Now this will work
explanations = lime_explainer.generate_explanations(sample_text)
lime_explainer.visualize_all_explanations(explanations)

def explain_with_lime(models, model_names, word2vec_model, text_samples,
                      y_true=None, num_features=5, num_samples=4):
    """
    Generate LIME explanations for multiple models with clear visualizations

    Args:
        models: List of trained models [log_reg, rf_model, svm_approx]
        model_names: List of model names for display
        word2vec_model: Loaded Word2Vec model
        text_samples: List of text samples to analyze
        y_true: Actual labels (optional)
        num_features: Top features to show per explanation
        num_samples: Number of samples to explain per model
    """

    # Initialize explainer with clear class names
    lime_explainer = LimeTextExplainer(
        class_names=["Non-Cyberbullying", "Cyberbullying"],
        verbose=False
    )

    # Prepare models - calibrate SVM if needed
    calibrated_models = []
```



```
for model in models:
    if hasattr(model, 'loss') and 'hinge' in str(model.loss): # Detect SGDClassifier
        calibrated = CalibratedClassifierCV(model, method='sigmoid')
        calibrated.fit(X_train, y_train)
        calibrated_models.append(calibrated)
    else:
        calibrated_models.append(model)

# Process each text sample
results = []
for i, text in enumerate(text_samples[:num_samples]):
    sample_results = {}

    # Store sample info
    sample_results['Text'] = text[:100] + "..." if len(text) > 100 else text
    if y_true is not None:
        sample_results['True_Label'] = y_true[i]

    # Explain with each model
    for model, name in zip(calibrated_models, model_names):
        # Vectorize and predict function
        def predict_fn(texts):
            vectors = np.array([vectorize_text(t, word2vec_model) for t in texts])
            return model.predict_proba(vectors.reshape(-1, 300))

        # Generate explanation
        explanation = lime_explainer.explain_instance(
            text,
            classifier_fn=predict_fn,
```

```
        num_features=num_features,
        labels=[0, 1] # Explain both classes
    )

    # Store results
    sample_results[name] = {
        'prediction': explanation.predict_proba.argmax(),
        'probability': explanation.predict_proba.max(),
        'features': dict(explanation.as_list())
    }

    results.append(sample_results)

return results

def vectorize_text(text, word2vec_model):
    """Convert text to word2vec embedding"""
    tokens = word_tokenize(text)
    words = [word for word in tokens if word in word2vec_model.wv]
    return np.mean(word2vec_model.wv[words], axis=0) if words else np.zeros(300)

def visualize_lime_results(results, model_names):
    """Create clear visualizations of LIME explanations"""
    for i, result in enumerate(results):
        plt.figure(figsize=(12, 6))

        # Create subplot for each model
        for j, model_name in enumerate(model_names):
            ax = plt.subplot(1, len(model_names), j+1)
```

```
# Get explanation data
features = list(result[model_name]['features'].keys())
weights = list(result[model_name]['features'].values())

# Sort by absolute weight
sorted_idx = np.argsort(np.abs(weights))[::-1]
features = [features[k] for k in sorted_idx]
weights = [weights[k] for k in sorted_idx]

# Plot with consistent colors
colors = ['#D81B60' if w > 0 else '#1E88E5' for w in weights]
y_pos = np.arange(len(features))

ax.barh(y_pos, weights, color=colors)
ax.set_yticks(y_pos)
ax.set_yticklabels(features)
ax.set_title(f'{model_name}\nP={result[model_name]['probability']:.2f}')
ax.set_xlabel("Feature Weight")
ax.grid(axis='x', alpha=0.2)

# Add legend only once
if j == 0:
    ax.legend(
        handles=[
            plt.Rectangle((0,0),1,1,color='#D81B60',label='Supports Cyberbullying'),
            plt.Rectangle((0,0),1,1,color='#1E88E5',label='Against Cyberbullying')
        ],
        loc='lower right'
```

```
)

plt.suptitle(f"Case {i+1}: {result['Text']}", y=1.02)
plt.tight_layout()
plt.show()

# Example Usage
text_samples = merged_df["Cleaned_Message"].sample(4).tolist()
true_labels      =      merged_df.loc[merged_df["Cleaned_Message"].isin(text_samples),
"Label"].tolist()

# Generate explanations
lime_results = explain_with_lime(
    models=[log_reg, rf_model, svm_approx],
    model_names=["Logistic Regression", "Random Forest", "SGD Classifier"],
    word2vec_model=word2vec_model,
    text_samples=text_samples,
    y_true=true_labels,
    num_features=5
)

"""Now I Will do SHap explanations for the 3 models."""

class SHAPExplainer:
    def __init__(self, models: List, model_names: List[str], word2vec_model, merged_df, X_test,
y_test, X_train=None, y_train=None):
        """
        Initialize SHAP explainer for multiple models

```

Args:

models: List of trained models [log_reg, rf_model, svm_approx]

model_names: List of model names

word2vec_model: Loaded Word2Vec model

merged_df: DataFrame containing text data

X_test: Test features

y_test: Test labels

X_train: Training features (required for calibrating SGD)

y_train: Training labels (required for calibrating SGD)

"""

self.models = models

self.model_names = model_names

self.word2vec_model = word2vec_model

self.merged_df = merged_df

self.X_test = X_test

self.y_test = y_test

self.X_train = X_train

self.y_train = y_train

self.embedding_dim = 300 # Dimension of Word2Vec embeddings

def get_sample_text(self, idx: int = 10) -> str:

"""Get a sample text from test data"""

test_indices = self.merged_df.index[self.merged_df["Label"].isin(self.y_test.tolist())]

return self.merged_df["Cleaned_Message"].iloc[test_indices[idx]]

def vectorize_text(self, text: str) -> np.ndarray:

"""Convert text to word2vec embedding"""

tokens = word_tokenize(text)

words = [word for word in tokens if word in self.word2vec_model.wv]

```
        return np.mean(self.word2vec_model.wv[words], axis=0) if words else  
np.zeros(self.embedding_dim)
```

```
def create_shap_explanations(self, sample_text: str) -> Dict[str, shap.Explanation]:  
    """Generate SHAP explanations for all models"""  
    explanations = {}  
    sample_vector = np.array([self.vectorize_text(sample_text)])  
    min_evals = 2 * self.embedding_dim + 1 # Minimum required evaluations  
  
    # Prepare models (calibrate SGD if needed)  
    calibrated_models = []  
    for model in self.models:  
        if hasattr(model, 'loss') and 'hinge' in str(model.loss): # Detect SGDClassifier  
            if self.X_train is None or self.y_train is None:  
                raise ValueError("X_train and y_train required for calibrating SGDClassifier")  
            calibrated = CalibratedClassifierCV(model, method='sigmoid')  
            calibrated.fit(self.X_train[:1000], self.y_train[:1000]) # Use subset for speed  
            calibrated_models.append(calibrated)  
        else:  
            calibrated_models.append(model)  
  
    # Generate explanations  
    for model, name in zip(calibrated_models, self.model_names):  
        try:  
            explainer = shap.Explainer(  
                lambda x: model.predict_proba(x.reshape(-1, self.embedding_dim)),  
                self.X_test[:100], # Background data  
                algorithm='auto'  
            )
```

```
        shap_values = explainer(
            sample_vector,
            max_evals=min(1000, min_evals + 200) # Balance speed/accuracy
        )
        explanations[name] = shap_values

    except Exception as e:
        print(f"Error explaining {name}: {str(e)}")
        explanations[name] = None

    return explanations
```

```
def generate_report(self, sample_text: str = None, top_n: int = 10):
    """Complete pipeline for generating and visualizing SHAP explanations"""
    if sample_text is None:
        sample_text = self.get_sample_text()

    print(f"Analyzing sample text:\n'{sample_text}'\n")
    print("="*80)

    explanations = self.create_shap_explanations(sample_text)
    self.plot_shap_explanations(explanations, sample_text, top_n)

    return explanations
```

Create Embedding Matrix

Enhancing Sentiments Analysis with Explainable AI Techniques

```
vocab_size = min(MAX_VOCAB_SIZE, len(tokenizer.word_index) + 1)
embedding_matrix = np.zeros((vocab_size, embedding_dim))

"""Import Required Libraries for BI LSTM"""

# Set hyperparameters
EMBEDDING_DIM = 100 # GloVe embedding size
MAX_SEQUENCE_LENGTH = 128 # Input sequence length
LSTM_UNITS = 128 # Number of LSTM units
DROPOUT_RATE = 0.3 # Dropout to prevent overfitting
LEARNING_RATE = 0.001 # Optimizer learning rate
BATCH_SIZE = 64 # Training batch size
EPOCHS = 5 # Number of training epochs

# Define Bi-LSTM Model
bi_lstm_model = Sequential([
    Embedding(input_dim=MAX_VOCAB_SIZE, output_dim=EMBEDDING_DIM,
              weights=[embedding_matrix], input_length=MAX_SEQUENCE_LENGTH,
              trainable=False),

    Bidirectional(LSTM(LSTM_UNITS, return_sequences=True)),
    Dropout(DROPOUT_RATE),

    Bidirectional(LSTM(LSTM_UNITS)),
    Dropout(DROPOUT_RATE),

    Dense(64, activation="relu"),
    Dense(1, activation="sigmoid") # Binary classification
])
```



```
# ♦ **Manually Build the Model**
```

```
bi_lstm_model.build(input_shape=(None, MAX_SEQUENCE_LENGTH))
```

```
# Compile model
```

```
bi_lstm_model.compile(  
    loss="binary_crossentropy",  
    optimizer=Adam(learning_rate=LEARNING_RATE),  
    metrics=["accuracy"]  
)
```

```
# Show model summary
```

```
bi_lstm_model.summary()
```

```
"""Training BI LSTM"""
```

```
# Define early stopping to prevent overfitting
```

```
early_stopping = tf.keras.callbacks.EarlyStopping(monitor='val_loss',  
                                                    patience=3,  
                                                    restore_best_weights=True)
```

```
# Train the model
```

```
history = bi_lstm_model.fit(  
    X_train, y_train,  
    validation_data=(X_test, y_test),  
    epochs=10,  
    batch_size=32,  
    callbacks=[early_stopping]  
)
```

```
"""Now Compiling, Training and Saving GRU"""
```

```
# Constants (make sure these are defined in your code)
MAX_VOCAB_SIZE = 10000 # Should match your tokenizer's num_words
EMBEDDING_DIM = 100    # Should match your GloVe embedding dimension
MAX_SEQUENCE_LENGTH = 128 # Should match your padded sequence length

# 1. Optimized Model Architecture
gru_model = Sequential([
    Embedding(
        input_dim=MAX_VOCAB_SIZE,
        output_dim=100,
        weights=[embedding_matrix],
        mask_zero=True,
        trainable=False # Frozen embeddings for speed
    ),

    # cuDNN-optimized GRU layers
    Bidirectional(
        GRU(32,
            return_sequences=True,
            kernel_regularizer=l2(0.001),
            activation='tanh', # Explicitly set for GPU optimization
            recurrent_activation='sigmoid'
        )
    ),

    GRU(16), # Single direction for speed

    Dense(16, activation='relu'),
```

```
Dense(1, activation='sigmoid')
])
```

2. Modern Optimizer Configuration

```
optimizer = Adam(
    learning_rate=0.005,
    global_clipnorm=1.0 # Gradient clipping
)
```

3. Compiled Model

```
gru_model.compile(
    loss='binary_crossentropy',
    optimizer=optimizer,
    metrics=['accuracy', tf.keras.metrics.AUC(name='auc')]
)
```

#4. Model Summary

```
gru_model.summary()
```

Explicitly build the model with input shape

```
gru_model.build(input_shape=(None, MAX_SEQUENCE_LENGTH))
```

Verify the model is built

```
print("\nModel is built:", gru_model.built) # Should print True
```

Check trainable parameters

```
print("Trainable params:", sum([tf.size(w).numpy() for w in gru_model.trainable_weights]))
print("Non-trainable params:", sum([tf.size(w).numpy() for w in gru_model.non_trainable_weights]))
```

```
# Hyperparameters for Speed
```

```
BATCH_SIZE = 512 # Maximize GPU utilization
```

```
EPOCHS = 15
```

```
# Data Pipeline Optimization
```

```
train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))
```

```
train_dataset = train_dataset.shuffle(10000).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

```
# Training Callbacks
```

```
callbacks = [
```

```
    tf.keras.callbacks.EarlyStopping(
```

```
        monitor='val_auc',
```

```
        patience=3,
```

```
        mode='max'
```

```
    )
```

```
]
```

```
# Start Training
```

```
history = gru_model.fit(
```

```
    train_dataset,
```

```
    validation_data=(X_test, y_test),
```

```
    epochs=EPOCHS,
```

```
    callbacks=callbacks,
```

```
    verbose=1
```

```
)
```

```
"""Now DistilBERT"""
```

```
!pip install transformers datasets evaluate accelerate -q
```

```
from sklearn.model_selection import train_test_split
```

```
import pandas as pd
```

```
# 1. Get your text data and labels
```

```
texts = merged_df["Message"].values # Raw text messages
```

```
labels = merged_df["Label"].values # Binary labels (0/1)
```

```
# 2. Split into train/test (keeping text format)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    texts,
```

```
    labels,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=labels # Important for imbalanced data
```

```
)
```

```
# 3. Convert to Dataset objects (required by HuggingFace)
```

```
from datasets import Dataset
```

```
train_dataset = Dataset.from_dict({"text": X_train, "label": y_train})
```

```
test_dataset = Dataset.from_dict({"text": X_test, "label": y_test})
```

```
print(f"Train samples: {len(train_dataset)}, Test samples: {len(test_dataset)}")
```

```
print(f"Sample train entry: {train_dataset[0]['text'][:50]}...")
```

```
from transformers import DistilBertTokenizer
```

```
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128 # Matches GRU's MAX_SEQUENCE_LENGTH
    )

# Apply tokenization
tokenized_train = train_dataset.map(tokenize_function, batched=True)
tokenized_test = test_dataset.map(tokenize_function, batched=True)

# Set format for PyTorch
tokenized_train.set_format("torch", columns=["input_ids", "attention_mask", "label"])
tokenized_test.set_format("torch", columns=["input_ids", "attention_mask", "label"])

print("\nTokenized example:")
print(tokenized_train[0].keys())
print(f"Input shape: {tokenized_train[0]['input_ids'].shape}")

from transformers import DistilBertForSequenceClassification, Trainer, TrainingArguments
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import torch

# 1. Initialize Model
model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased',
```

```
num_labels=2,  
id2label={0: "Non-Cyberbullying", 1: "Cyberbullying"},  
label2id={"Non-Cyberbullying": 0, "Cyberbullying": 1}  
)to("cuda" if torch.cuda.is_available() else "cpu")
```

2. Define Metrics

```
def compute_metrics(pred):
```

```
    labels = pred.label_ids  
    preds = pred.predictions.argmax(-1)  
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary')  
    acc = accuracy_score(labels, preds)  
    return {  
        'accuracy': acc,  
        'f1': f1,  
        'precision': precision,  
        'recall': recall  
    }
```

3. Set Training Arguments (Optimized for Colab T4)

```
training_args = TrainingArguments(  
    output_dir='./results',  
    num_train_epochs=3,  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=32,  
    evaluation_strategy="epoch",  
    save_strategy="epoch",  
    logging_steps=50,  
    fp16=True, # Enable mixed precision  
    load_best_model_at_end=True,
```

```
metric_for_best_model='f1',  
greater_is_better=True  
)
```

4. Initialize Trainer

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_train,  
    eval_dataset=tokenized_test,  
    compute_metrics=compute_metrics  
)
```


Implementation File 2 Code Snippets

(Preloading Models, Embeddings & Machine Learning Train-Test Splits)

```
"""#Full Evaluation of Models

#Now I will do LIME Explanations for Logistic Regression, SGD Classifier and Random Forest.
"""

class LimeExplainer:
    def __init__(self, models, model_names, word2vec_model, merged_df, y_test, X_train=None,
y_train=None):
        self.models = models
        self.model_names = model_names
        self.word2vec_model = word2vec_model
        self.merged_df = merged_df
        self.y_test = y_test
        self.X_train = X_train
        self.y_train = y_train
        self.explainer = lime.lime_text.LimeTextExplainer(
            class_names=["0", "1"], # Match your label encoding
            verbose=True
        )

    def get_sample_text(self, idx=10):
        """Get a sample text from test data"""
        test_indices = self.merged_df.index[self.merged_df["Label"].isin(self.y_test.tolist())]
        return self.merged_df["Cleaned_Message"].iloc[test_indices[idx]]
```

```
def vectorize_text(self, text):
    """Convert text to word2vec embedding"""
    tokens = word_tokenize(text)
    words = [word for word in tokens if word in self.word2vec_model.wv]
    return np.mean(self.word2vec_model.wv[words], axis=0) if words else np.zeros(300)

def predict_proba_wrapper(self, model, texts):
    """Wrapper that ensures consistent label ordering"""
    vectorized_texts = np.array([self.vectorize_text(text) for text in texts])
    probs = model.predict_proba(vectorized_texts.reshape(-1, 300))

    # Ensure probabilities are for classes [0, 1]
    if probs.shape[1] == 2: # Binary classification
        return probs
    else: # Handle potential single-column output
        return np.column_stack([1-probs, probs])

def generate_explanations(self, sample_text, num_features=10):
    """Generate explanations with label consistency checks"""
    explanations = []
    for model in self.models:
        try:
            # Calibrate if needed (for SGD)
            if hasattr(model, 'loss') and 'hinge' in str(model.loss) and self.X_train is not None:
                model = CalibratedClassifierCV(model, method='sigmoid').fit(self.X_train,
self.y_train)

            exp = self.explainer.explain_instance(
                sample_text,
```

```
        lambda x: self.predict_proba_wrapper(model, x),
        num_features=num_features,
        labels=[0, 1] # Explicitly specify expected labels
    )
    explanations.append(exp)
except Exception as e:
    print(f"Error explaining {type(model).__name__}: {str(e)}")
    explanations.append(None)
return explanations

def plot_lime_explanation(self, exp, model_name):
    """Robust plotting with updated LIME API"""
    if exp is None:
        print(f"Skipping {model_name} - no explanation generated")
        return

    try:
        # Get explanation for class 1 (Cyberbullying) - no num_features parameter
        words_weights = exp.as_list(label=1)
        words, weights = zip(*words_weights)

        plt.figure(figsize=(10, 5))
        colors = ['#D81B60' if w > 0 else '#1E88E5' for w in weights]
        bars = plt.barh(words, weights, color=colors)

        # Add prediction info
        prob = exp.predict_proba[1]
        plt.title(f"{model_name}\nP(Cyberbullying)={prob:.2f}", fontsize=14)
        plt.xlabel("Feature Weight", fontsize=12)
```

```
plt.ylabel("Words", fontsize=12)
plt.gca().invert_yaxis()

# Add legend
plt.legend(
    [plt.Rectangle((0,0),1,1,color=c) for c in ['#D81B60', '#1E88E5']],
    ['Supports Cyberbullying', 'Against Cyberbullying'],
    loc='lower right'
)
plt.tight_layout()
plt.show()

except Exception as e:
    print(f"Error plotting {model_name}: {str(e)}")

def visualize_all_explanations(self, explanations):
    """Visualize all explanations without num_features"""
    for exp, name in zip(explanations, self.model_names):
        self.plot_lime_explanation(exp, name)

# Usage Example
if __name__ == "__main__":
    # Initialize with your loaded models and data
    lime_explainer = LimeExplainer(
        models=[log_reg, rf_model, svm_approx],
        model_names=["Logistic Regression", "Random Forest", "SGD Classifier"],
        word2vec_model=word2vec_model,
        merged_df=merged_df,
        y_test=y_test,
```

```
X_train=X_train, # Needed for SGD calibration
y_train=y_train # Needed for SGD calibration
)

# Generate and view explanations
sample_text = lime_explainer.get_sample_text(idx=10)
explanations = lime_explainer.generate_explanations(sample_text)
lime_explainer.visualize_all_explanations(explanations)

"""#Now I will test different cases on the 3 machine learning model to compare their
behaviour"""

import pandas as pd
from lime.lime_text import LimeTextExplainer
from sklearn.calibration import CalibratedClassifierCV
from nltk.tokenize import word_tokenize

def explain_with_lime(models, model_names, word2vec_model, text_samples,
                      y_true=None, num_features=5, num_samples=4):
    """
    Generate LIME explanations for multiple models with clear visualizations

    Args:
        models: List of trained models [log_reg, rf_model, svm_approx]
        model_names: List of model names for display
        word2vec_model: Loaded Word2Vec model
        text_samples: List of text samples to analyze
        y_true: Actual labels (optional)
        num_features: Top features to show per explanation
```

```
num_samples: Number of samples to explain per model
"""

# Initialize explainer with clear class names
lime_explainer = LimeTextExplainer(
    class_names=["Non-Cyberbullying", "Cyberbullying"],
    verbose=False
)

# Prepare models - calibrate SVM if needed
calibrated_models = []
for model in models:
    if hasattr(model, 'loss') and 'hinge' in str(model.loss): # Detect SGDClassifier
        calibrated = CalibratedClassifierCV(model, method='sigmoid')
        calibrated.fit(X_train, y_train)
        calibrated_models.append(calibrated)
    else:
        calibrated_models.append(model)

# Process each text sample
results = []
for i, text in enumerate(text_samples[:num_samples]):
    sample_results = {}

    # Store sample info
    sample_results['Text'] = text[:100] + "..." if len(text) > 100 else text
    if y_true is not None:
        sample_results['True_Label'] = y_true[i]

    # Explain with each model
```

```
for model, name in zip(calibrated_models, model_names):
    # Vectorize and predict function
    def predict_fn(texts):
        vectors = np.array([vectorize_text(t, word2vec_model) for t in texts])
        return model.predict_proba(vectors.reshape(-1, 300))

    # Generate explanation
    explanation = lime_explainer.explain_instance(
        text,
        classifier_fn=predict_fn,
        num_features=num_features,
        labels=[0, 1] # Explain both classes
    )

    # Store results
    sample_results[name] = {
        'prediction': explanation.predict_proba.argmax(),
        'probability': explanation.predict_proba.max(),
        'features': dict(explanation.as_list())
    }

    results.append(sample_results)

return results

def vectorize_text(text, word2vec_model):
    """Convert text to word2vec embedding"""
    tokens = word_tokenize(text)
    words = [word for word in tokens if word in word2vec_model.wv]
```

```
return np.mean(word2vec_model.wv[words], axis=0) if words else np.zeros(300)

def visualize_lime_results(results, model_names):
    """Create clear visualizations of LIME explanations"""
    for i, result in enumerate(results):
        plt.figure(figsize=(12, 6))

        # Create subplot for each model
        for j, model_name in enumerate(model_names):
            ax = plt.subplot(1, len(model_names), j+1)

            # Get explanation data
            features = list(result[model_name]['features'].keys())
            weights = list(result[model_name]['features'].values())

            # Sort by absolute weight
            sorted_idx = np.argsort(np.abs(weights))[::-1]
            features = [features[k] for k in sorted_idx]
            weights = [weights[k] for k in sorted_idx]

            # Plot with consistent colors
            colors = ['#D81B60' if w > 0 else '#1E88E5' for w in weights]
            y_pos = np.arange(len(features))

            ax.barh(y_pos, weights, color=colors)
            ax.set_yticks(y_pos)
            ax.set_yticklabels(features)
            ax.set_title(f"{model_name}\nP={result[model_name]['probability']:.2f}")
            ax.set_xlabel("Feature Weight")
```



```
ax.grid(axis='x', alpha=0.2)

# Add legend only once
if j == 0:
    ax.legend(
        handles=[
            plt.Rectangle((0,0),1,1,color='#D81B60',label='Supports Cyberbullying'),
            plt.Rectangle((0,0),1,1,color='#1E88E5',label='Against Cyberbullying')
        ],
        loc='lower right'
    )

plt.suptitle(f"Case {i+1}: {result['Text']}", y=1.02)
plt.tight_layout()
plt.show()

# Example Usage
text_samples = merged_df["Cleaned_Message"].sample(4).tolist()
true_labels      =      merged_df.loc[merged_df["Cleaned_Message"].isin(text_samples),
"Label"].tolist()

# Generate explanations
lime_results = explain_with_lime(
    models=[log_reg, rf_model, svm_approx],
    model_names=["Logistic Regression", "Random Forest", "SGD Classifier"],
    word2vec_model=word2vec_model,
    text_samples=text_samples,
    y_true=true_labels,
    num_features=5
```

```
)
```

```
# Visualize results
```

```
visualize_lime_results(lime_results, ["Logistic Regression", "Random Forest", "SGD Classifier"])
```

```
"""#Now I will use SHap on the 3 Machine Learning Models"""
```

```
class SHAPExplainer:
```

```
    def __init__(self, models: List, model_names: List[str], word2vec_model, merged_df, X_test,
y_test, X_train=None, y_train=None):
```

```
        """
```

```
        Initialize SHAP explainer for multiple models
```

```
        Args:
```

```
        models: List of trained models [log_reg, rf_model, svm_approx]
```

```
        model_names: List of model names
```

```
        word2vec_model: Loaded Word2Vec model
```

```
        merged_df: DataFrame containing text data
```

```
        X_test: Test features
```

```
        y_test: Test labels
```

```
        X_train: Training features (required for calibrating SGD)
```

```
        y_train: Training labels (required for calibrating SGD)
```

```
        """
```

```
        self.models = models
```

```
        self.model_names = model_names
```

```
        self.word2vec_model = word2vec_model
```

```
        self.merged_df = merged_df
```

```
        self.X_test = X_test
```

```
        self.y_test = y_test
```

```
self.X_train = X_train
self.y_train = y_train
self.embedding_dim = 300 # Dimension of Word2Vec embeddings

def get_sample_text(self, idx: int = 10) -> str:
    """Get a sample text from test data"""
    test_indices = self.merged_df.index[self.merged_df["Label"].isin(self.y_test.tolist())]
    return self.merged_df["Cleaned_Message"].iloc[test_indices[idx]]

def vectorize_text(self, text: str) -> np.ndarray:
    """Convert text to word2vec embedding"""
    tokens = word_tokenize(text)
    words = [word for word in tokens if word in self.word2vec_model.wv]
    return np.mean(self.word2vec_model.wv[words], axis=0) if words else
np.zeros(self.embedding_dim)

def create_shap_explanations(self, sample_text: str) -> Dict[str, shap.Explanation]:
    """Generate SHAP explanations for all models"""
    explanations = {}
    sample_vector = np.array([self.vectorize_text(sample_text)])
    min_evals = 2 * self.embedding_dim + 1 # Minimum required evaluations

    # Prepare models (calibrate SGD if needed)
    calibrated_models = []
    for model in self.models:
        if hasattr(model, 'loss') and 'hinge' in str(model.loss): # Detect SGDClassifier
            if self.X_train is None or self.y_train is None:
                raise ValueError("X_train and y_train required for calibrating SGDClassifier")
            calibrated = CalibratedClassifierCV(model, method='sigmoid')
```

```
        calibrated.fit(self.X_train[:1000], self.y_train[:1000]) # Use subset for speed
        calibrated_models.append(calibrated)
    else:
        calibrated_models.append(model)

# Generate explanations
for model, name in zip(calibrated_models, self.model_names):
    try:
        explainer = shap.Explainer(
            lambda x: model.predict_proba(x.reshape(-1, self.embedding_dim)),
            self.X_test[:100], # Background data
            algorithm='auto'
        )

        shap_values = explainer(
            sample_vector,
            max_evals=min(1000, min_evals + 200) # Balance speed/accuracy
        )
        explanations[name] = shap_values

    except Exception as e:
        print(f"Error explaining {name}: {str(e)}")
        explanations[name] = None

return explanations

def plot_shap_explanations(self, explanations: Dict[str, shap.Explanation], sample_text: str,
top_n: int = 10):
    """Visualize SHAP explanations for all models"""
```

```
words = word_tokenize(sample_text)

for name, shap_values in explanations.items():
    if shap_values is None:
        print(f"Skipping {name} - no explanation generated")
        continue

    try:
        # Process SHAP values for positive class (Cyberbullying)
        vals = shap_values.values[0, :, 1] # Class 1 (Cyberbullying)
        shap_dict = dict(zip(words, vals))
        sorted_features = sorted(shap_dict.items(),
                                key=lambda x: abs(x[1]),
                                reverse=True)[:top_n]

        # Create plot
        plt.figure(figsize=(10, 5))
        features, contributions = zip(*sorted_features)

        # Corrected color scheme (red for supports cyberbullying)
        colors = ['#D81B60' if val > 0 else '#1E88E5' for val in contributions]

        bars = plt.barh(features, contributions, color=colors)

        # Add prediction probability
        pred_proba = shap_values.base_values[0][1] # Base value for class 1
        plt.title(f"SHAP Explanation - {name}\nP(Cyberbullying) = {pred_proba:.2f}",
                  fontsize=14, pad=20)
```

```
plt.xlabel("SHAP Value (Impact on Prediction)", fontsize=12)
plt.ylabel("Words", fontsize=12)
plt.gca().invert_yaxis()

# Corrected legend
plt.legend(
    handles=[
        plt.Rectangle((0,0), 1, 1, color='#D81B60', label='Supports Cyberbullying'),
        plt.Rectangle((0,0), 1, 1, color='#1E88E5', label='Against Cyberbullying')
    ],
    loc='lower right',
    framealpha=1
)

# Value annotations
for bar in bars:
    width = bar.get_width()
    plt.text(width/2, bar.get_y() + bar.get_height()/2,
             f"{width:.4f}",
             va='center', ha='center',
             color='white' if abs(width) > 0.02 else 'black')

plt.grid(axis='x', alpha=0.2)
plt.tight_layout()
plt.show()

except Exception as e:
    print(f"Error plotting {name}: {str(e)}")
```

```
def generate_report(self, sample_text: str = None, top_n: int = 10):  
    """Complete pipeline for generating and visualizing SHAP explanations"""  
    if sample_text is None:  
        sample_text = self.get_sample_text()  
  
    print(f"Analyzing sample text:\n'{sample_text}'\n")  
    print("="*80)  
  
    explanations = self.create_shap_explanations(sample_text)  
    self.plot_shap_explanations(explanations, sample_text, top_n)  
  
    return explanations
```

Usage Example

```
if __name__ == "__main__":  
    # Initialize with your loaded models and data  
    shap_explainer = SHAPExplainer(  
        models=[log_reg, rf_model, svm_approx],  
        model_names=["Logistic Regression", "Random Forest", "SGD Classifier"],  
        word2vec_model=word2vec_model,  
        merged_df=merged_df,  
        X_test=X_test,  
        y_test=y_test,  
        X_train=X_train, # Required for SGD calibration  
        y_train=y_train # Required for SGD calibration  
    )  
  
    # Generate and display explanations  
    explanations = shap_explainer.generate_report(  

```

```
sample_text="youre so stupid nobody likes you get a life you pathetic person", # Optional:  
specify text  
top_n=8 # Number of features to show  
)
```

```
"""#Now I will Apply LIME on the 3 Deep Learning Models (Bi LSTM, GRU, DistilBERT)"""
```

```
import numpy as np  
import matplotlib.pyplot as plt  
from lime.lime_text import LimeTextExplainer  
import tensorflow as tf  
from tensorflow.keras.preprocessing.sequence import pad_sequences  
import torch  
from transformers import DistilBertTokenizer  
from nltk.tokenize import word_tokenize  
import re
```

```
class DeepLIMEExplainer:
```

```
def __init__(self, bi_lstm_model, gru_model, distilbert_model, distilbert_tokenizer,  
             word_index=None, max_len=128):
```

```
    """
```

```
    LIME explainer for deep learning models (BiLSTM, GRU, DistilBERT)
```

```
    Args:
```

```
        bi_lstm_model: Trained BiLSTM model
```

```
        gru_model: Trained GRU model
```

```
        distilbert_model: Trained DistilBERT model
```

```
        distilbert_tokenizer: HuggingFace tokenizer for DistilBERT
```

```
        word_index: Dictionary mapping words to indices for Keras models
```



```
        max_len: Maximum sequence length
        """

        self.models = [bi_lstm_model, gru_model, distilbert_model]
        self.model_names = ["BiLSTM", "GRU", "DistilBERT"]
        self.distilbert_tokenizer = distilbert_tokenizer
        self.word_index = word_index
        self.max_len = max_len
        self.explainer = LimeTextExplainer(
            class_names=["Not Bullying", "Bullying"],
            split_expression="\s+", # Split on whitespace
            bow=False, # Don't use bag of words
            mask_string="" # Empty string for masked tokens
        )

    def _text_to_sequences(self, texts):
        """Convert texts to sequences using word_index"""
        sequences = []
        for text in texts:
            words = word_tokenize(text.lower())
            seq = [self.word_index.get(word, 0) for word in words if word in self.word_index]
            sequences.append(seq)
        return sequences

    def _predict_keras_model(self, model, texts):
        """Prediction function for Keras models (BiLSTM, GRU)"""
        # Pre-process text
        sequences = self._text_to_sequences(texts)
        padded = pad_sequences(sequences, maxlen=self.max_len)
```

```
# Make predictions
preds = model.predict(padded, verbose=0)

# Ensure output is in format [p(not_bullying), p(bullying)]
if preds.shape[1] == 1: # Binary output
    return np.column_stack([1-preds, preds])
return preds

def _predict_distilbert(self, texts):
    """Prediction function for DistilBERT model"""
    # Tokenize inputs
    inputs = self.distilbert_tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=self.max_len
    )

    # Make predictions
    with torch.no_grad():
        outputs = self.models[2](**inputs)
        logits = outputs.logits
        probs = torch.softmax(logits, dim=-1).numpy()

    return probs

def explain_instance(self, text, model_index=0, num_features=10, num_samples=1000):
    """
```

Generate LIME explanation for a specific model

Args:

text: Text to explain

model_index: 0=BiLSTM, 1=GRU, 2=DistilBERT

num_features: Number of features to show

num_samples: Number of samples for LIME

Returns:

LIME explanation object

"""

Select model and prediction function

if model_index in [0, 1]: # BiLSTM or GRU

 predict_fn = lambda texts: self._predict_keras_model(self.models[model_index], texts)

else: # DistilBERT

 predict_fn = self._predict_distilbert

Generate explanation

exp = self.explainer.explain_instance(

 text,

 predict_fn,

 num_features=num_features,

 num_samples=num_samples,

 labels=[1] # Explain bullying class

)

return exp

```
# Filter by minimum count
filtered_words = [word for word, count in word_counts.items() if count >= min_count]

# Create word_index (1-indexed, 0 reserved for padding)
word_index = {word: i+1 for i, word in enumerate(filtered_words)}

return word_index

"""#Now I will Apply SHap on the 3 Deep Learning Models (Bi LSTM, GRU, DistilBERT)"""

class DeepSHAPExplainer:
    def __init__(self, bi_lstm_model, gru_model, distilbert_model, distilbert_tokenizer,
                 word_index=None, max_len=128, background_data=None):
        """
        SHAP explainer for deep learning models (BiLSTM, GRU, DistilBERT)

        Args:
            bi_lstm_model: Trained BiLSTM model
            gru_model: Trained GRU model
            distilbert_model: Trained DistilBERT model
            distilbert_tokenizer: HuggingFace tokenizer for DistilBERT
            word_index: Dictionary mapping words to indices for Keras models
            max_len: Maximum sequence length
            background_data: Background dataset for SHAP (can be X_train subset)
        """
        self.models = [bi_lstm_model, gru_model, distilbert_model]
```

```
self.model_names = ["BiLSTM", "GRU", "DistilBERT"]
self.distilbert_tokenizer = distilbert_tokenizer
self.word_index = word_index
self.max_len = max_len
self.background_data = background_data

# Initialize SHAP explainers for each model
self.explainers = {}
self._initialize_explainers()

def _initialize_explainers(self):
    """Initialize SHAP explainers for each model"""
    if self.background_data is not None:
        try:
            # Process background data for Keras models (using a small subset)
            background_texts = self.background_data[:10] # Start with a small number

            # For BiLSTM
            sequences = self._text_to_sequences(background_texts)
            padded = pad_sequences(sequences, maxlen=self.max_len)

            # Convert to proper tensor format expected by TF models
            background_data_tensor = tf.convert_to_tensor(padded, dtype=tf.float32)

            # BiLSTM SHAP explainer
            self.explainers["BiLSTM"] = shap.DeepExplainer(
                model=self.models[0],
                data=background_data_tensor
            )
```

```
# GRU SHAP explainer
self.explainers["GRU"] = shap.DeepExplainer(
    model=self.models[1],
    data=background_data_tensor
)

except Exception as e:
    print(f"Error initializing deep explainers: {str(e)}")
    # We'll fall back to Kernel explainer in explain_instance

def _text_to_sequences(self, texts):
    """Convert texts to sequences using word_index"""
    sequences = []
    for text in texts:
        words = word_tokenize(text.lower())
        seq = [self.word_index.get(word, 0) for word in words if word in self.word_index]
        sequences.append(seq)
    return sequences

def _predict_keras_model(self, model, texts):
    """Prediction function for Keras models (BiLSTM, GRU)"""
    # Pre-process text
    sequences = self._text_to_sequences(texts)
    padded = pad_sequences(sequences, maxlen=self.max_len)

    # Make predictions
    preds = model.predict(padded, verbose=0)
```

```
# Ensure output is in format [p(not_bullying), p(bullying)]
if preds.shape[1] == 1: # Binary output
    return np.column_stack([1-preds, preds])
return preds

def _predict_distilbert(self, texts):
    """Prediction function for DistilBERT model"""
    # Tokenize inputs
    inputs = self.distilbert_tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=self.max_len
    )

    # Make predictions
    with torch.no_grad():
        outputs = self.models[2](**inputs)
        logits = outputs.logits
        probs = torch.softmax(logits, dim=-1).numpy()

    return probs

def explain_keras_model(self, text, model_index):
    """Simpler explanation method that should be more reliable"""
    model = self.models[model_index]
    model_name = self.model_names[model_index]
```

```
# Tokenize text
words = word_tokenize(text.lower())
words = words[:30] # Limit to first 30 words for performance

# Get base prediction with all words
sequence = self._text_to_sequences([text])[0]
padded = pad_sequences([sequence], maxlen=self.max_len)
base_pred = model.predict(padded, verbose=0)

# Get probability of bullying
base_prob = base_pred[0][1] if base_pred.shape[1] > 1 else base_pred[0][0]

# Calculate impact of each word by removing it
word_shap = {}
for word in words:
    # Create text without this word
    modified_text = ' '.join([w for w in words if w != word])
    if not modified_text:
        modified_text = "[empty]"

    # Get prediction
    mod_sequence = self._text_to_sequences([modified_text])[0]
    mod_padded = pad_sequences([mod_sequence], maxlen=self.max_len)
    mod_pred = model.predict(mod_padded, verbose=0)

    # Get probability of bullying
    mod_prob = mod_pred[0][1] if mod_pred.shape[1] > 1 else mod_pred[0][0]

    # SHAP value is how much prediction changes when word is removed
```



```
word_shap[word] = base_prob - mod_prob

return word_shap

def explain_distilbert(self, text, num_samples=100):
    """Generate SHAP explanation for DistilBERT model"""
    try:
        # Tokenize input
        tokenized = self.distilbert_tokenizer(
            text,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=self.max_len
        )

        # Create a wrapper for the model
        def predict_fn(texts):
            return self._predict_distilbert(texts)

        # Get prediction for the text
        pred = predict_fn([text])[0]

        # Get input IDs and convert to text
        input_ids = tokenized['input_ids'][0].tolist()
        tokens = self.distilbert_tokenizer.convert_ids_to_tokens(input_ids)

        # Create word presence dictionary
        word_presence = {token: 1 for token in tokens if token not in ['[CLS]', '[SEP]', '[PAD]']}
```

```
# Generate more samples by randomly removing words
samples = []
for _ in range(num_samples):
    # Create a random sample by keeping some words
    sample = {token: (1 if np.random.random() > 0.5 else 0)
               for token in tokens if token not in ['[CLS]', '[SEP]', '[PAD]']}
    samples.append(sample)

# Convert samples to text
sample_texts = []
for sample in samples:
    # Convert to detokenized text
    sample_tokens = [token for token in tokens if sample.get(token, 0) == 1
                     or token in ['[CLS]', '[SEP]']]
    sample_text = self.distilbert_tokenizer.convert_tokens_to_string(sample_tokens)
    sample_texts.append(sample_text if sample_text else "[empty]")

# Get predictions for samples
predictions = predict_fn(sample_texts)

# Create a kernel explainer
explainer = shap.KernelExplainer(
    lambda x: np.array([pred_fn([sample_text])[0][1] for sample_text in x]),
    sample_texts
)

# Get SHAP values
shap_values = explainer.shap_values(text)
```

```
# Map SHAP values to words
word_shap = {}
words = word_tokenize(text.lower())
for word in words:
    word_shap[word] = shap_values[words.index(word)] if word in words else 0

return word_shap

except Exception as e:
    print(f"Error explaining DistilBERT: {str(e)}")

# Fall back to using a simpler approach
try:
    # Create predictor function
    def predict_fn(text_list):
        return self._predict_distilbert(text_list)

    # Tokenize the input again
    words = word_tokenize(text.lower())

    # Get base prediction
    base_pred = predict_fn([text])[0][1] # Bullying probability

    # Compute importance by removing each word
    word_shap = {}
    for word in words:
        # Create text without this word
        modified_text = ' '.join([w for w in words if w != word])
```

```
    if not modified_text:
        modified_text = "[empty]"

    # Get prediction for modified text
    mod_pred = predict_fn([modified_text])[0][1]

    # SHAP value is how much prediction changes when word is removed
    word_shap[word] = base_pred - mod_pred

    return word_shap

except Exception as e2:
    print(f"Error with fallback approach for DistilBERT: {str(e2)}")
    return {}

def explain_instance(self, text, model_index=0):
    """
    Generate SHAP explanation for a specific model

    Args:
        text: Text to explain
        model_index: 0=BiLSTM, 1=GRU, 2=DistilBERT

    Returns:
        Dictionary mapping words to SHAP values
    """
    if model_index < 2: # BiLSTM or GRU
        return self.explain_keras_model(text, model_index)
    else: # DistilBERT
```

```
        return self.explain_distilbert(text)

def visualize_explanation(self, word_shap, model_name, max_words=15):
    """Visualize SHAP explanation"""
    if not word_shap:
        print(f"No explanation available for {model_name}")
        return

    # Sort by absolute value
    sorted_words = sorted(word_shap.items(), key=lambda x: abs(x[1]), reverse=True)

    # Take top words
    top_words = sorted_words[:max_words]
    words, values = zip(*top_words)

    # Create plot
    plt.figure(figsize=(10, 6))

    # Use consistent color scheme
    colors = ['#D81B60' if v > 0 else '#1E88E5' for v in values]

    # Plot bars
    plt.barh(words, values, color=colors)

    # Add title
    plt.title(f"{model_name} - SHAP Explanation", fontsize=14)
    plt.xlabel("SHAP Value (Impact on Prediction)", fontsize=12)
    plt.grid(axis='x', alpha=0.3)
    plt.gca().invert_yaxis()
```

```
# Add legend
plt.legend(
    handles=[
        plt.Rectangle((0,0), 1, 1, color='#D81B60', label='Supports Bullying'),
        plt.Rectangle((0,0), 1, 1, color='#1E88E5', label='Against Bullying')
    ],
    loc='lower right'
)

plt.tight_layout()
plt.show()

def explain_all_models(self, text, max_words=10):
    """Generate and visualize explanations for all models"""
    plt.figure(figsize=(18, 6))

    for i, model_name in enumerate(self.model_names):
        try:
            # Generate explanation
            word_shap = self.explain_instance(text, model_index=i)

            if not word_shap:
                raise ValueError(f"No explanation generated for {model_name}")

            # Sort by absolute value
            sorted_words = sorted(word_shap.items(), key=lambda x: abs(x[1]), reverse=True)

            # Take top words
```

```
top_words = sorted_words[:max_words]
words, values = zip(*top_words)

# Create subplot
plt.subplot(1, 3, i+1)

# Use consistent color scheme
colors = ['#D81B60' if v > 0 else '#1E88E5' for v in values]

# Plot bars
plt.barh(words, values, color=colors)

# Add prediction info
pred_prob = self._get_model_prediction(text, i)
plt.title(f"{model_name}\nP(Bullying)={pred_prob:.2f}", fontsize=12)
plt.xlabel("SHAP Value", fontsize=10)
plt.grid(axis='x', alpha=0.3)
plt.gca().invert_yaxis()

# Add legend only for the first subplot
if i == 0:
    plt.legend(
        handles=[
            plt.Rectangle((0,0), 1, 1, color='#D81B60', label='Supports Bullying'),
            plt.Rectangle((0,0), 1, 1, color='#1E88E5', label='Against Bullying')
        ],
        loc='lower right'
    )
```

```
except Exception as e:
    plt.subplot(1, 3, i+1)
    plt.text(0.5, 0.5, f"Error: {str(e)}",
            ha='center', va='center', fontsize=10, wrap=True)
    plt.axis('off')
    print(f"Error explaining {model_name}: {str(e)}")

plt.suptitle(f"SHAP Analysis: '{text[:50]}...', fontsize=14, y=1.05)
plt.tight_layout()
plt.show()

def _get_model_prediction(self, text, model_index):
    """Get prediction probability for a model"""
    if model_index < 2: # BiLSTM or GRU
        preds = self._predict_keras_model(self.models[model_index], [text])
        return preds[0][1] # Probability of class 1 (bullying)
    else: # DistilBERT
        preds = self._predict_distilbert([text])
        return preds[0][1] # Probability of class 1 (bullying)

def highlight_text_with_shap(self, text, model_index=0):
    """Generate HTML with highlighted text based on SHAP values"""
    # Get explanation
    word_shap = self.explain_instance(text, model_index)

    if not word_shap:
        return "<p>No explanation available</p>"

    # Tokenize text
```



```
words = word_tokenize(text)

# Create HTML with highlights
html = []
for word in words:
    weight = word_shap.get(word.lower(), 0)

    # Skip words with no weight
    if abs(weight) < 0.01:
        html.append(word)
        continue

    # Define color intensity based on weight
    intensity = min(abs(weight) * 5, 1.0) # Scale weight for better visibility

    # Choose color based on positive/negative contribution
    if weight > 0:
        color = f"rgba(216, 27, 96, {intensity})" # Red for positive contribution
    else:
        color = f"rgba(30, 136, 229, {intensity})" # Blue for negative contribution

    html.append(f"<span style='background-color: {color};'>{word}</span>")

# Join with spaces
html_text = ' '.join(html)

# Add prediction info
model_name = self.model_names[model_index]
pred_prob = self._get_model_prediction(text, model_index)
```

```
return f"""
    <div style="font-family: Arial, sans-serif; padding: 15px; border: 1px solid #ddd; border-
radius: 5px;">
        <h3>{model_name} - SHAP Analysis</h3>
        <p>Prediction: {pred_prob:.2f} (Bullying probability)</p>
        <p>{html_text}</p>
        <div style="margin-top: 10px;">
            <span style="background-color: rgba(216, 27, 96, 0.3); padding: 3px;">Red = Supports
Bullying</span>
            <span style="background-color: rgba(30, 136, 229, 0.3); padding: 3px; margin-left:
10px;">Blue = Against Bullying</span>
        </div>
    </div>
    """
```

```
"""#Now I will Apply Attention Mechanism DistilBERT"""
```

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.preprocessing.sequence import pad_sequences
import torch
from transformers import DistilBertTokenizer
from nltk.tokenize import word_tokenize
import matplotlib.cm as cm
from IPython.display import HTML, display
import re
from scipy.special import softmax
```

```
class AttentionVisualizer:
```

```
    def __init__(self, bi_lstm_model, gru_model, distilbert_model, distilbert_tokenizer,  
                  word_index=None, max_len=128):
```

```
        self.models = [bi_lstm_model, gru_model, distilbert_model]
```

```
        self.model_names = ["BiLSTM", "GRU", "DistilBERT"]
```

```
        self.distilbert_tokenizer = distilbert_tokenizer
```

```
        self.word_index = word_index
```

```
        self.max_len = max_len
```

```
        # Initialize attention extractors
```

```
        self.bilstm_compute_attention = None
```

```
        self.gru_compute_attention = None
```

```
        # Get vocabulary size from word_index
```

```
        if self.word_index is not None:
```

```
            self.vocab_size = max(self.word_index.values()) + 1
```

```
            print(f"Vocabulary size from word_index: {self.vocab_size}")
```

```
        else:
```

```
            self.vocab_size = 30000 # Increased default size
```

```
            print("No word_index provided, using default vocab size of 30000")
```

```
        # Initialize models before using them
```

```
        self._initialize_models()
```

```
        # Create attention extractors
```

```
        self._create_attention_extractors()
```

```
    def _initialize_models(self):
```

```
"""Initialize models with dummy input"""
dummy_input = np.zeros((1, self.max_len), dtype=np.int32)

# Try to initialize BiLSTM model
if self.models[0] is not None:
    try:
        print("Initializing BiLSTM model...")
        self.models[0](dummy_input)
        print("BiLSTM model successfully initialized")
    except Exception as e:
        print(f"Error initializing BiLSTM model: {e}")
        print("Will try alternative initialization...")
        try:
            # Alternative approach - call predict
            self.models[0].predict(dummy_input, verbose=0)
            print("BiLSTM model initialized via predict()")
        except Exception as e2:
            print(f"Alternative initialization also failed: {e2}")

# Try to initialize GRU model
if self.models[1] is not None:
    try:
        print("Initializing GRU model...")
        self.models[1](dummy_input)
        print("GRU model successfully initialized")
    except Exception as e:
        print(f"Error initializing GRU model: {e}")
        print("Will try alternative initialization...")
        try:
```

```
# Alternative approach - call predict
self.models[1].predict(dummy_input, verbose=0)
print("GRU model initialized via predict()")
except Exception as e2:
    print(f"Alternative initialization also failed: {e2}")

def _create_attention_extractors(self):
    """Create attention extractors for BiLSTM and GRU"""
    self.bilstm_compute_attention = self._create_keras_attention_extractor(
        self.models[0], "BiLSTM"
    )
    self.gru_compute_attention = self._create_keras_attention_extractor(
        self.models[1], "GRU"
    )

    if self.bilstm_compute_attention is None:
        print("Could not create attention extractor for BiLSTM model.")
    if self.gru_compute_attention is None:
        print("Could not create attention extractor for GRU model.")

def _create_keras_attention_extractor(self, model, model_type):
    """Creates a function to compute attention for a Keras model"""
    try:
        # Find the embedding layer to determine max index
        embedding_layer = None
        for layer in model.layers:
            if isinstance(layer, tf.keras.layers.Embedding):
                embedding_layer = layer
                break
```

```
elif hasattr(layer, 'layers'):
    # Check if this is a container layer (like Sequential)
    for inner_layer in layer.layers:
        if isinstance(inner_layer, tf.keras.layers.Embedding):
            embedding_layer = inner_layer
            break
```

1. LIME Analysis

```
print("\n1. LIME Analysis")
try:
    exp = lime_explainer.explain_instance(text, model_index=model_index)
    lime_explainer.visualize_explanation(exp, model_name)
except Exception as e:
    print(f"LIME analysis failed: {str(e)}")
```

2. SHAP Analysis

```
print("\n2. SHAP Analysis")
try:
    word_shap = shap_explainer.explain_instance(text, model_index=model_index)
    shap_explainer.visualize_explanation(word_shap, model_name)
except Exception as e:
    print(f"SHAP analysis failed: {str(e)}")
```

3. Attention Analysis

```
print("\n3. Attention Analysis")
try:
    attention_visualizer.visualize_attention(text, model_index=model_index)
except Exception as e:
    print(f"Attention analysis failed: {str(e)}")
```

```
# HTML displays with word highlighting
print("\n4. Text Highlighting")

# LIME highlighting
try:
    lime_html = lime_explainer.highlight_text_with_lime(text, model_index)
    display(HTML(lime_html))
except Exception as e:
    print(f"LIME highlighting failed: {str(e)}")

# SHAP highlighting
try:
    shap_html = shap_explainer.highlight_text_with_shap(text, model_index)
    display(HTML(shap_html))
except Exception as e:
    print(f"SHAP highlighting failed: {str(e)}")

# Attention highlighting
try:
    attention_html = attention_visualizer.highlight_text_with_attention(text, model_index)
    display(HTML(attention_html))
except Exception as e:
    print(f"Attention highlighting failed: {str(e)}")
```